US 20250285391A1

(19) **United States**

(12) **Patent Application Publication**

(10) **Pub. No.: US 2025/0285391 A1**

(43) **Pub. Date: Sep. 11, 2025**

Goodrich et al.

(54) **VIRTUAL SURFACE MODIFICATION**

(71) Applicant: **Snap Inc.**, Santa Monica, CA (US)

(72) Inventors: **Kyle Goodrich**, Venice, CA (US); **Samuel Edward Hare**, Los Angeles, CA (US); **Maxim Maximov Lazarov**, Culver City, CA (US); **Tony Mathew**, Irvine, CA (US); **Andrew James McPhee**, Culver City, CA (US); **Wentao Shang**, Los Angeles, CA (US)

(21) Appl. No.: **19/216,016**

(22) Filed: **May 22, 2025**

Related U.S. Application Data

(63) Continuation of application No. 17/963,090, filed on Oct. 10, 2022, which is a continuation of application No. 16/723,540, filed on Dec. 20, 2019, now Pat. No. 11,501,499.

(60) Provisional application No. 62/782,916, filed on Dec. 20, 2018.

Publication Classification

(51) **Int. Cl.**

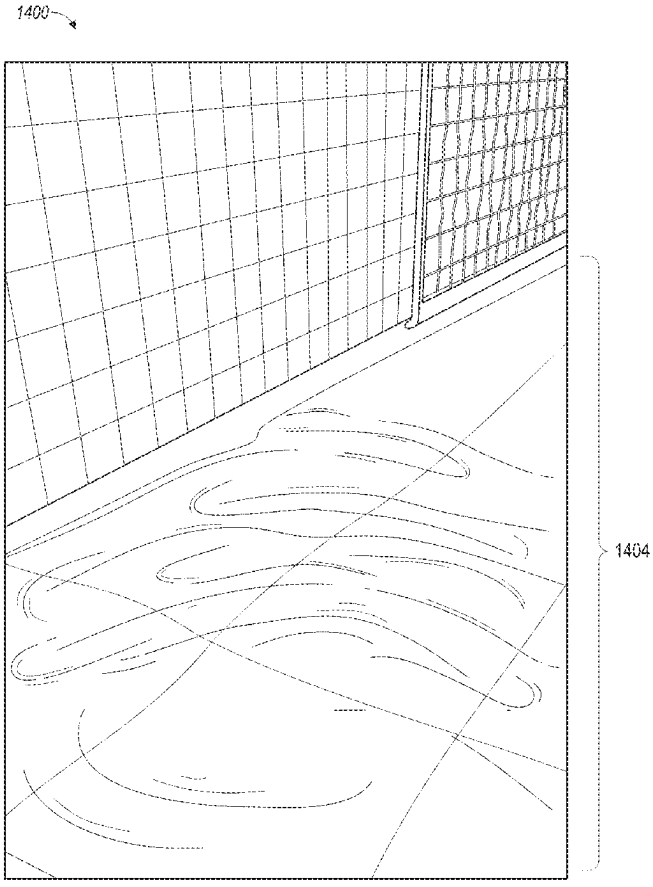
G06T 19/00	(2011.01)
G06F 3/01	(2006.01)
G06T 7/11	(2017.01)
G06T 7/194	(2017.01)
G06T 15/20	(2011.01)
G06T 19/20	(2011.01)

(52) **U.S. Cl.**

CPC ***G06T 19/006*** (2013.01); ***G06F 3/011*** (2013.01); ***G06T 7/11*** (2017.01); ***G06T 7/194*** (2017.01); ***G06T 15/205*** (2013.01); ***G06T 19/20*** (2013.01); *G06T 2207/10016* (2013.01)

(57) **ABSTRACT**

Aspects of the present disclosure involve a system comprising a computer-readable storage medium storing at least one program and a method for rendering virtual modifications to real-world environments depicted in image content. A reference surface is detected in a three-dimensional (3D) space captured within a camera feed produced by a camera of a computing device. An image mask is applied to the reference surface within the 3D space captured within the camera feed. A visual effect is applied to the image mask corresponding to the reference surface in the 3D space. The application of the visual effect to the image mask causes a modified surface to be rendered in presenting the camera feed on a display of the computing device.



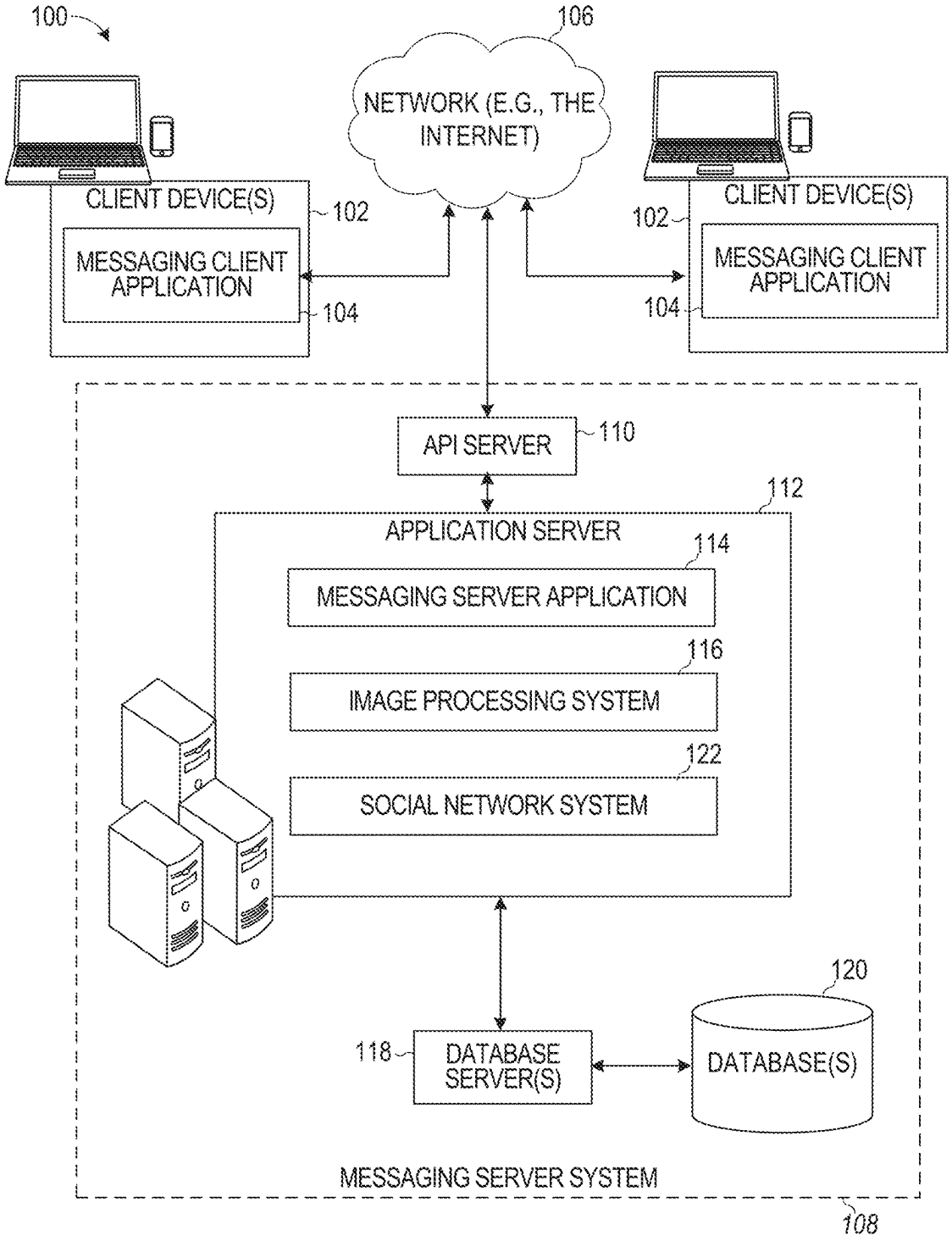

100
106
NETWORK (E.G., THE INTERNET)
CLIENT DEVICE(S)
102
MESSAGING CLIENT APPLICATION
104
CLIENT DEVICE(S)
102
MESSAGING CLIENT APPLICATION
104
API SERVER
110
112
APPLICATION SERVER
114
MESSAGING SERVER APPLICATION
116
IMAGE PROCESSING SYSTEM
122
SOCIAL NETWORK SYSTEM
120
118
DATABASE SERVER(S)
DATABASE(S)
MESSAGING SERVER SYSTEM
108

FIG. 1

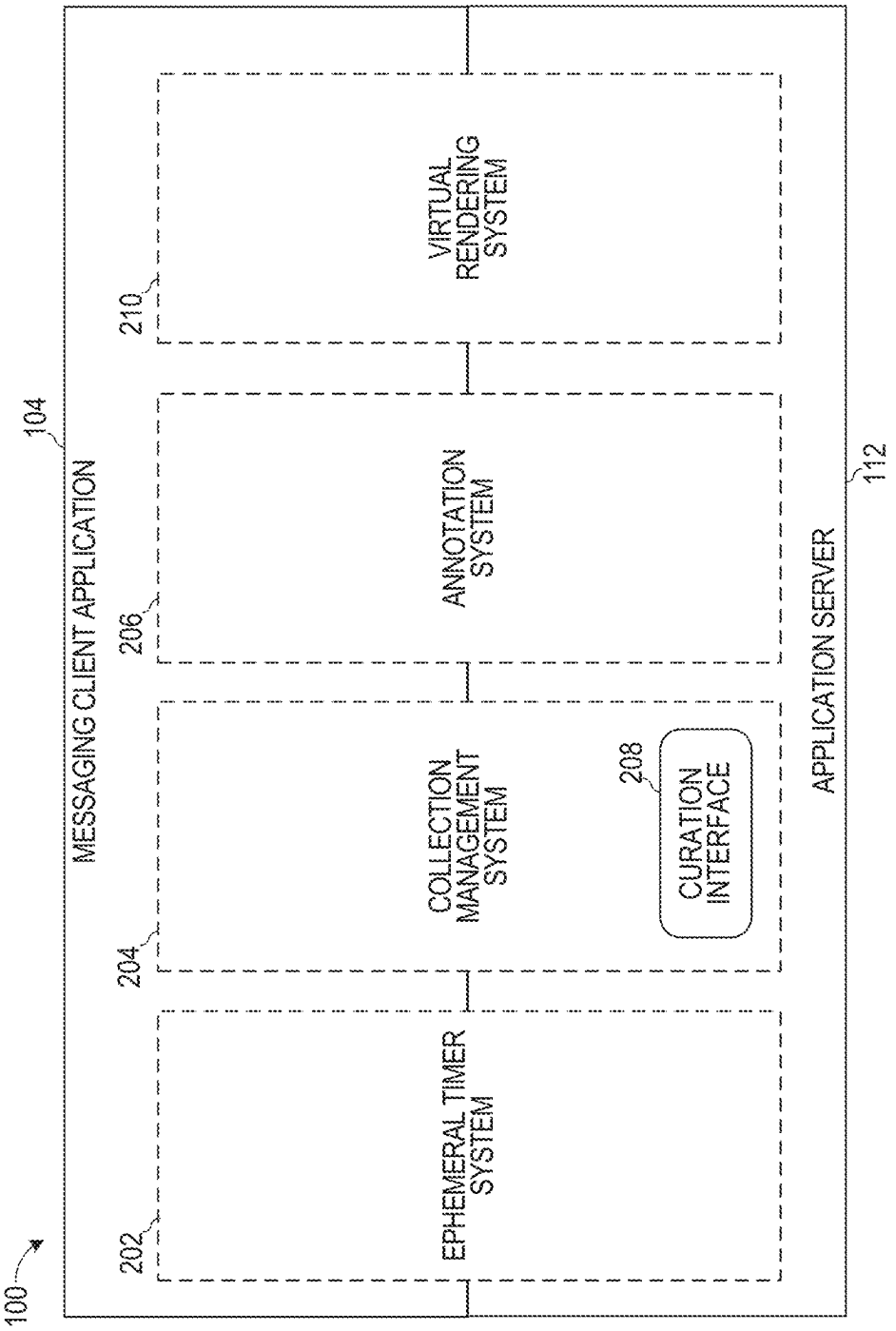

100
104
MESSAGING CLIENT APPLICATION
202
EPHEMERAL TIMER SYSTEM
204
COLLECTION MANAGEMENT SYSTEM
208
CURATION INTERFACE
206
ANNOTATION SYSTEM
210
VIRTUAL RENDERING SYSTEM
112
APPLICATION SERVER

FIG. 2

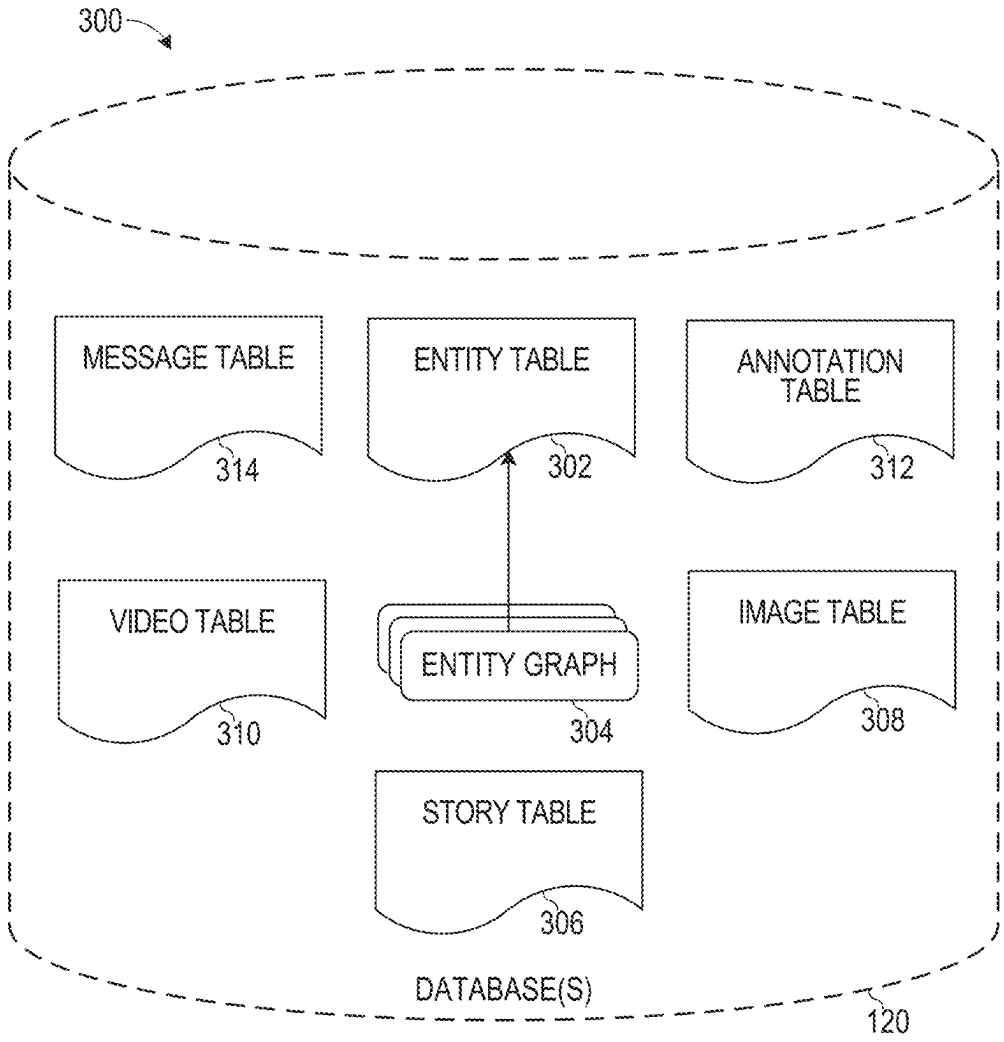

300
MESSAGE TABLE
314
ENTITY TABLE
302
ANNOTATION TABLE
312
VIDEO TABLE
310
ENTITY GRAPH
304
IMAGE TABLE
308
STORY TABLE
306
DATABASE(S)
120

FIG. 3

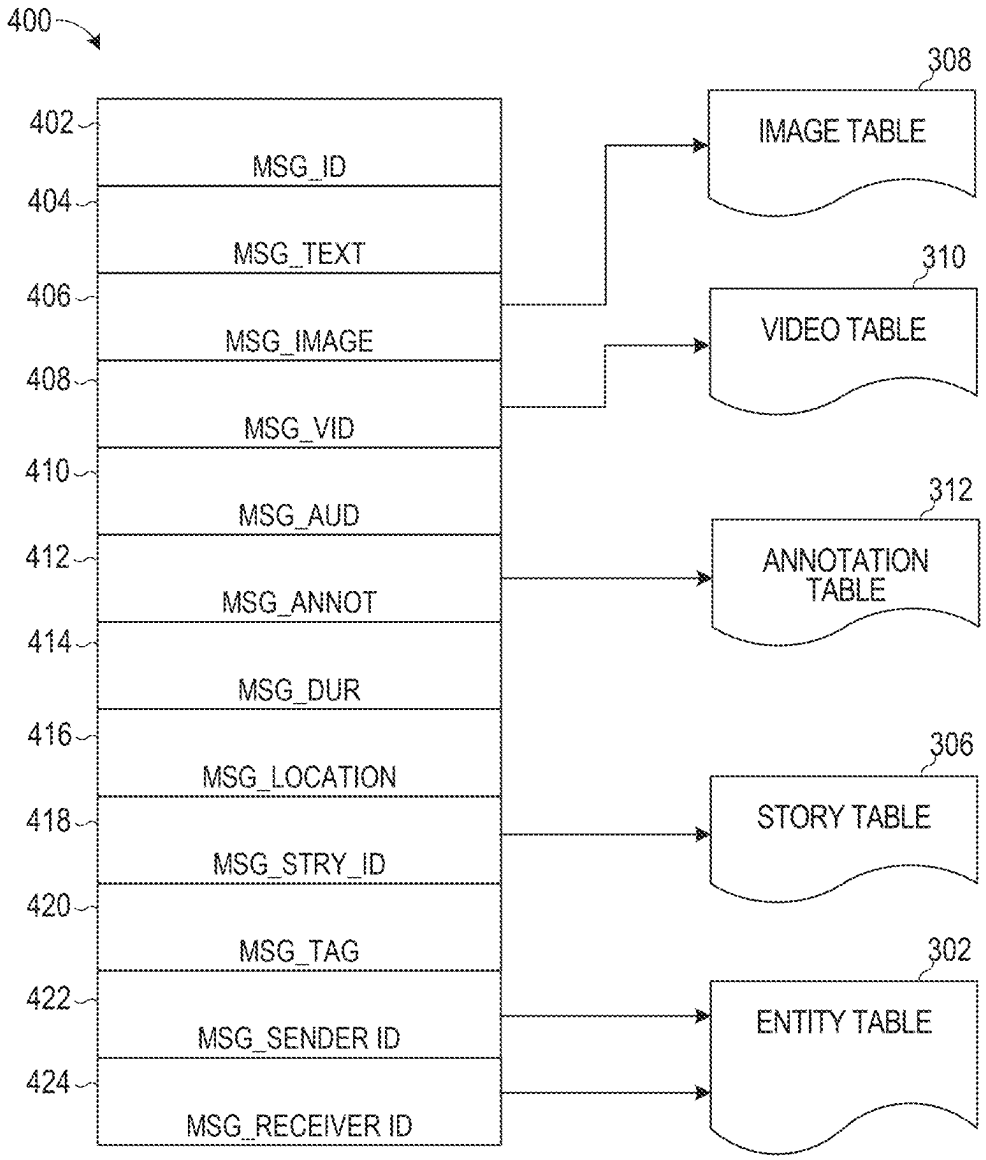

400
402
MSG_ID
404
MSG_TEXT
406
MSG_IMAGE
408
MSG_VID
410
MSG_AUD
412
MSG_ANNOT
414
MSG_DUR
416
MSG_LOCATION
418
MSG_STRY_ID
420
MSG_TAG
422
MSG_SENDER ID
424
MSG_RECEIVER ID
308
IMAGE TABLE
310
VIDEO TABLE
312
ANNOTATION TABLE
306
STORY TABLE
302
ENTITY TABLE

FIG. 4

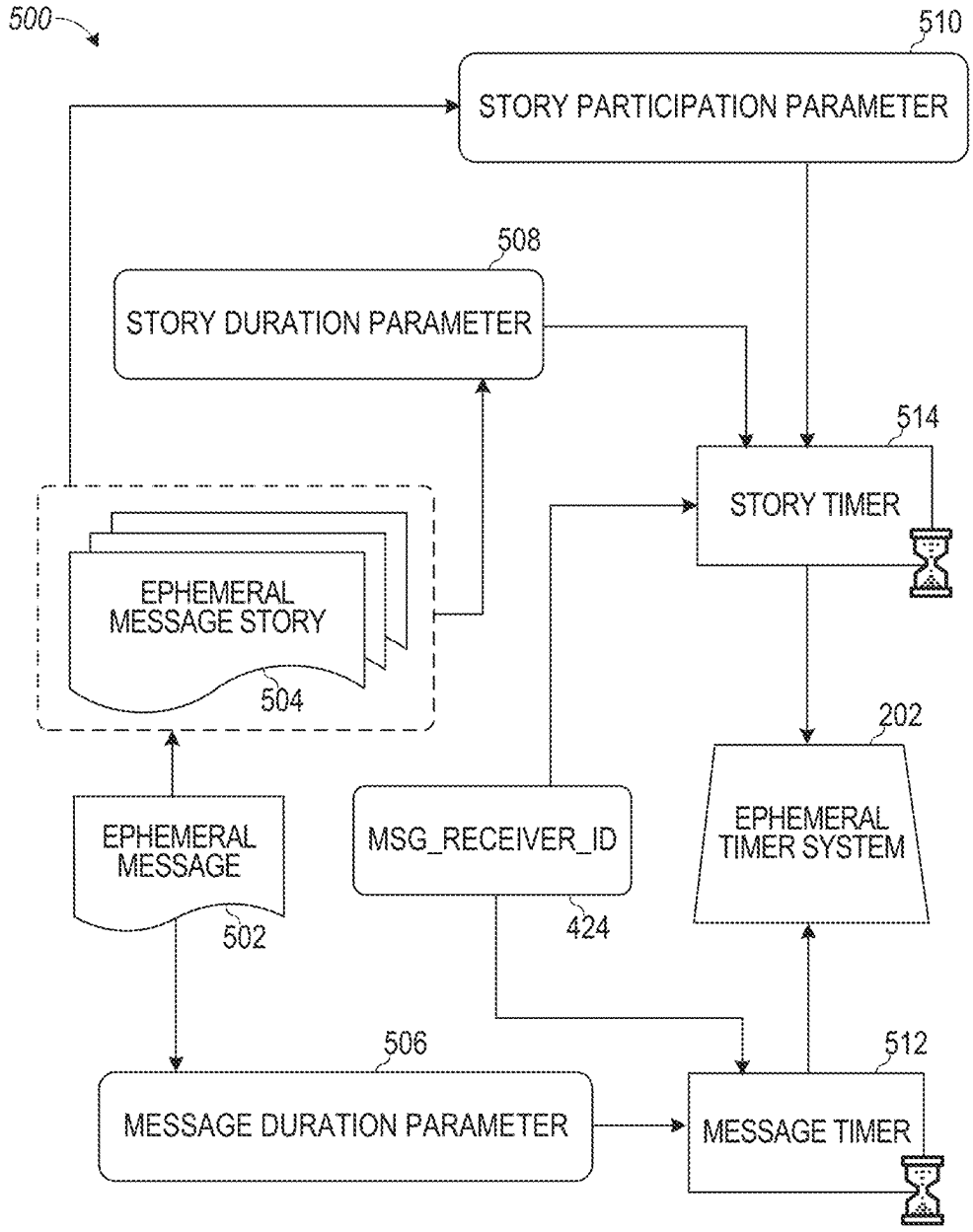

500
510
STORY PARTICIPATION PARAMETER
508
STORY DURATION PARAMETER
514
STORY TIMER
EPHEMERAL MESSAGE STORY
504
202
EPHEMERAL TIMER SYSTEM
EPHEMERAL MESSAGE
502
MSG_RECEIVER_ID
424
506
512
MESSAGE DURATION PARAMETER
MESSAGE TIMER

FIG. 5

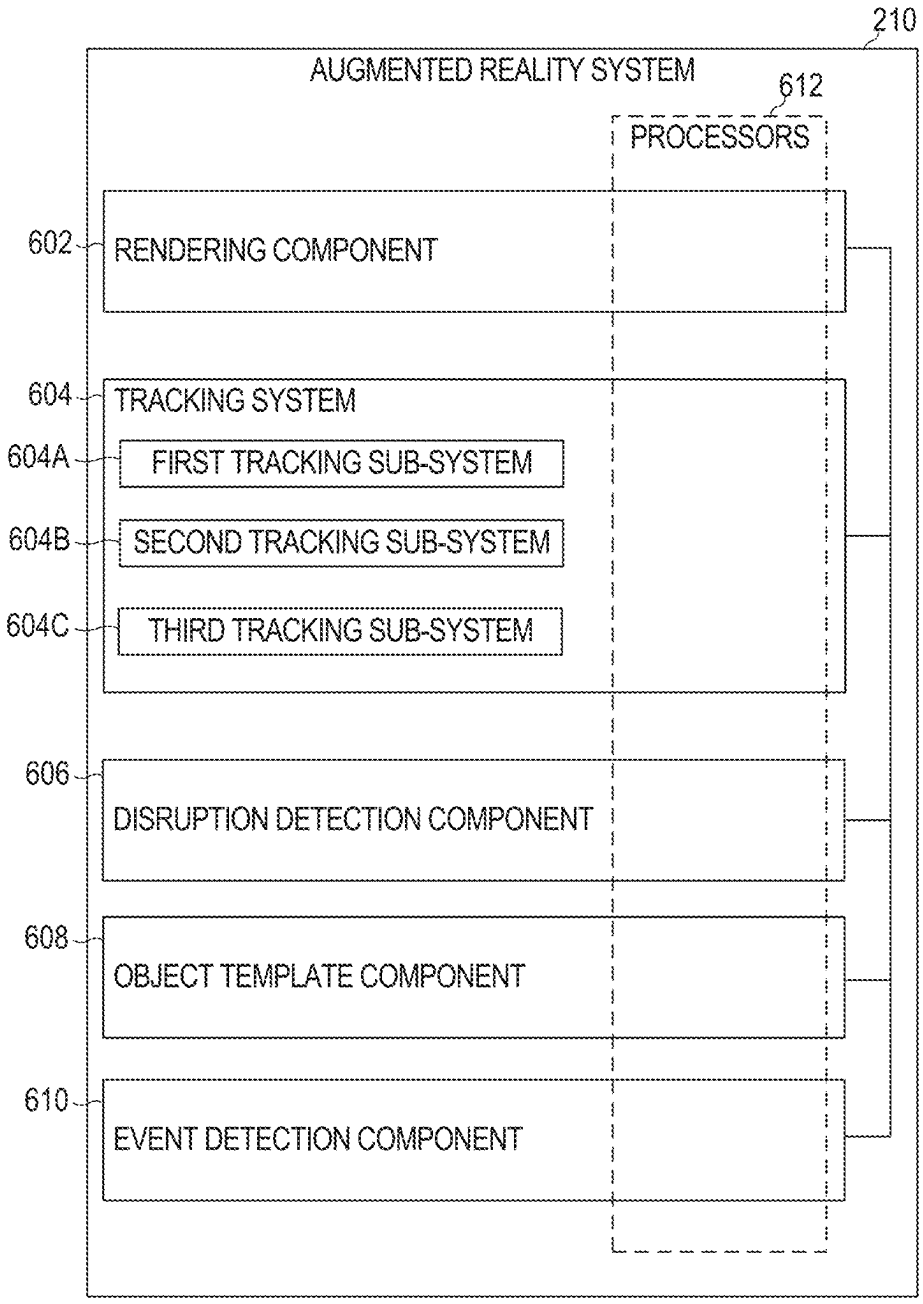

210
AUGMENTED REALITY SYSTEM
612
PROCESSORS
602
RENDERING COMPONENT
604
TRACKING SYSTEM
604A
FIRST TRACKING SUB-SYSTEM
604B
SECOND TRACKING SUB-SYSTEM
604C
THIRD TRACKING SUB-SYSTEM
606
DISRUPTION DETECTION COMPONENT
608
OBJECT TEMPLATE COMPONENT
610
EVENT DETECTION COMPONENT

FIG. 6

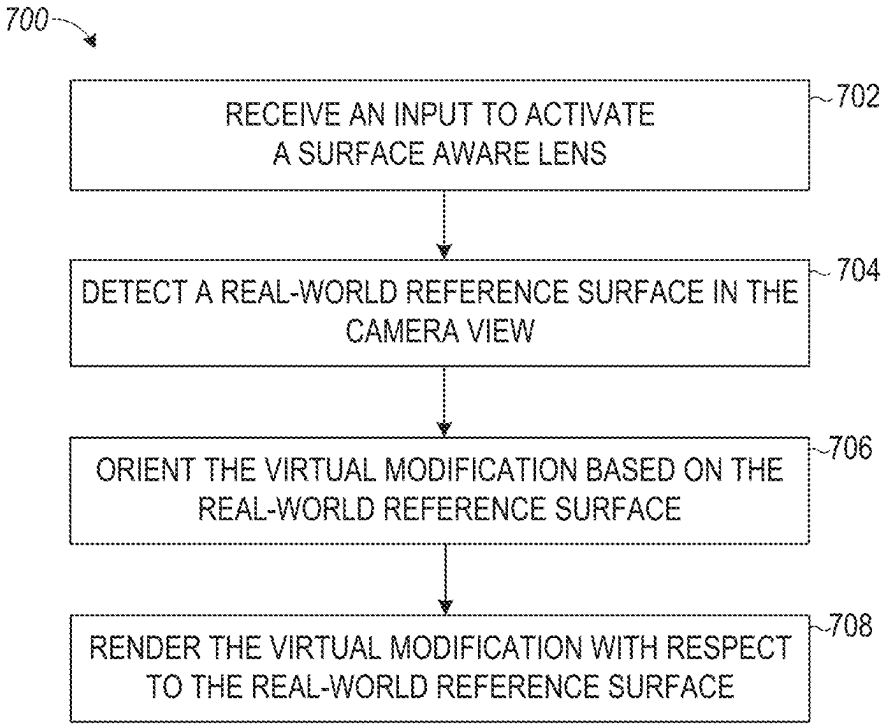

700
RECEIVE AN INPUT TO ACTIVATE A SURFACE AWARE LENS
702
DETECT A REAL-WORLD REFERENCE SURFACE IN THE CAMERA VIEW
704
ORIENT THE VIRTUAL MODIFICATION BASED ON THE REAL-WORLD REFERENCE SURFACE
706
RENDER THE VIRTUAL MODIFICATION WITH RESPECT TO THE REAL-WORLD REFERENCE SURFACE
708

FIG. 7

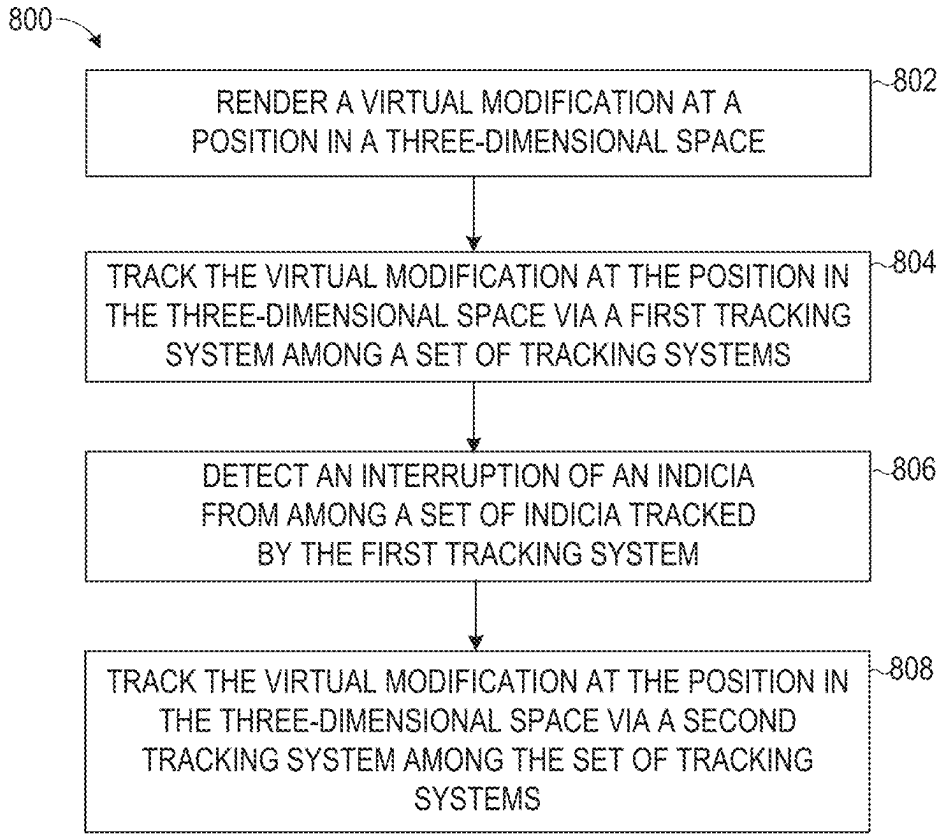

800
RENDER A VIRTUAL MODIFICATION AT A POSITION IN A THREE-DIMENSIONAL SPACE
802
TRACK THE VIRTUAL MODIFICATION AT THE POSITION IN THE THREE-DIMENSIONAL SPACE VIA A FIRST TRACKING SYSTEM AMONG A SET OF TRACKING SYSTEMS
804
DETECT AN INTERRUPTION OF AN INDICIA FROM AMONG A SET OF INDICIA TRACKED BY THE FIRST TRACKING SYSTEM
806
TRACK THE VIRTUAL MODIFICATION AT THE POSITION IN THE THREE-DIMENSIONAL SPACE VIA A SECOND TRACKING SYSTEM AMONG THE SET OF TRACKING SYSTEMS
808

FIG. 8

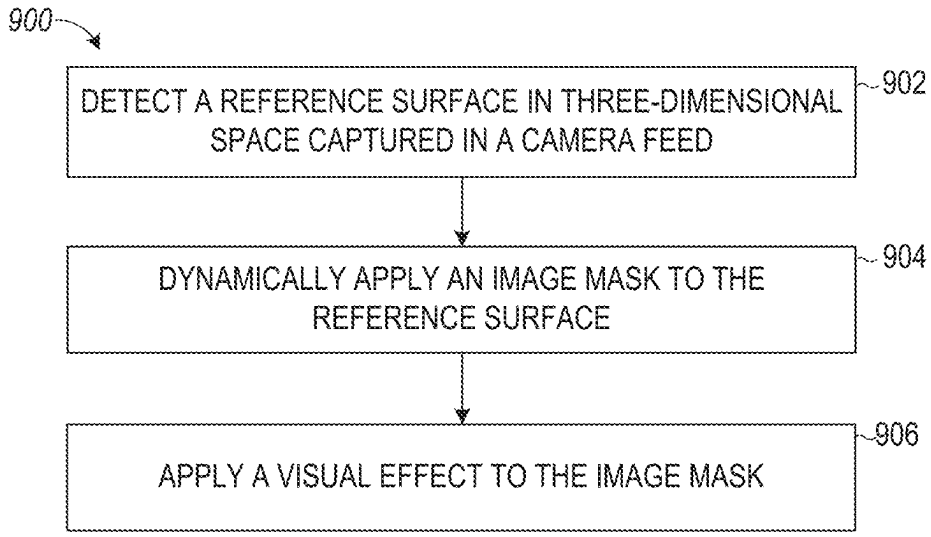

900
DETECT A REFERENCE SURFACE IN THREE-DIMENSIONAL SPACE CAPTURED IN A CAMERA FEED
902
DYNAMICALLY APPLY AN IMAGE MASK TO THE REFERENCE SURFACE
904
APPLY A VISUAL EFFECT TO THE IMAGE MASK
906

FIG. 9

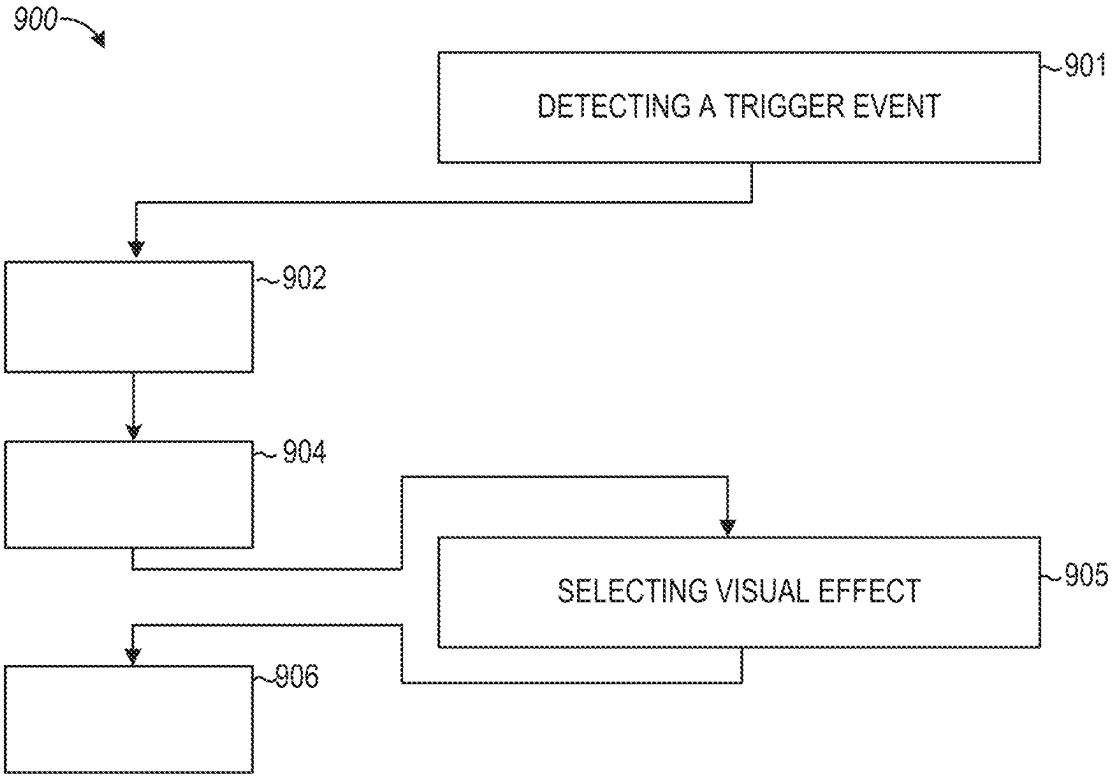

900
DETECTING A TRIGGER EVENT
901
902
904
SELECTING VISUAL EFFECT
905
906

FIG. 10

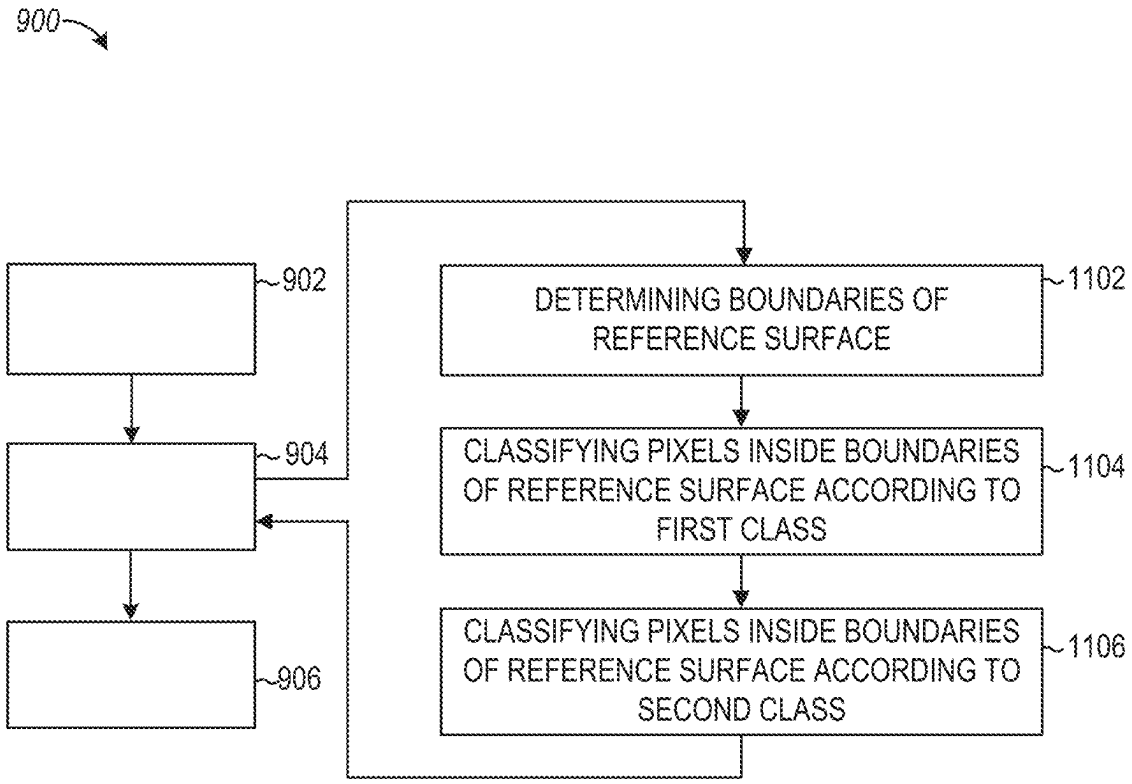

900
902
904
906
DETERMINING BOUNDARIES OF REFERENCE SURFACE
1102
CLASSIFYING PIXELS INSIDE BOUNDARIES OF REFERENCE SURFACE ACCORDING TO FIRST CLASS
1104
CLASSIFYING PIXELS INSIDE BOUNDARIES OF REFERENCE SURFACE ACCORDING TO SECOND CLASS
1106

FIG. 11

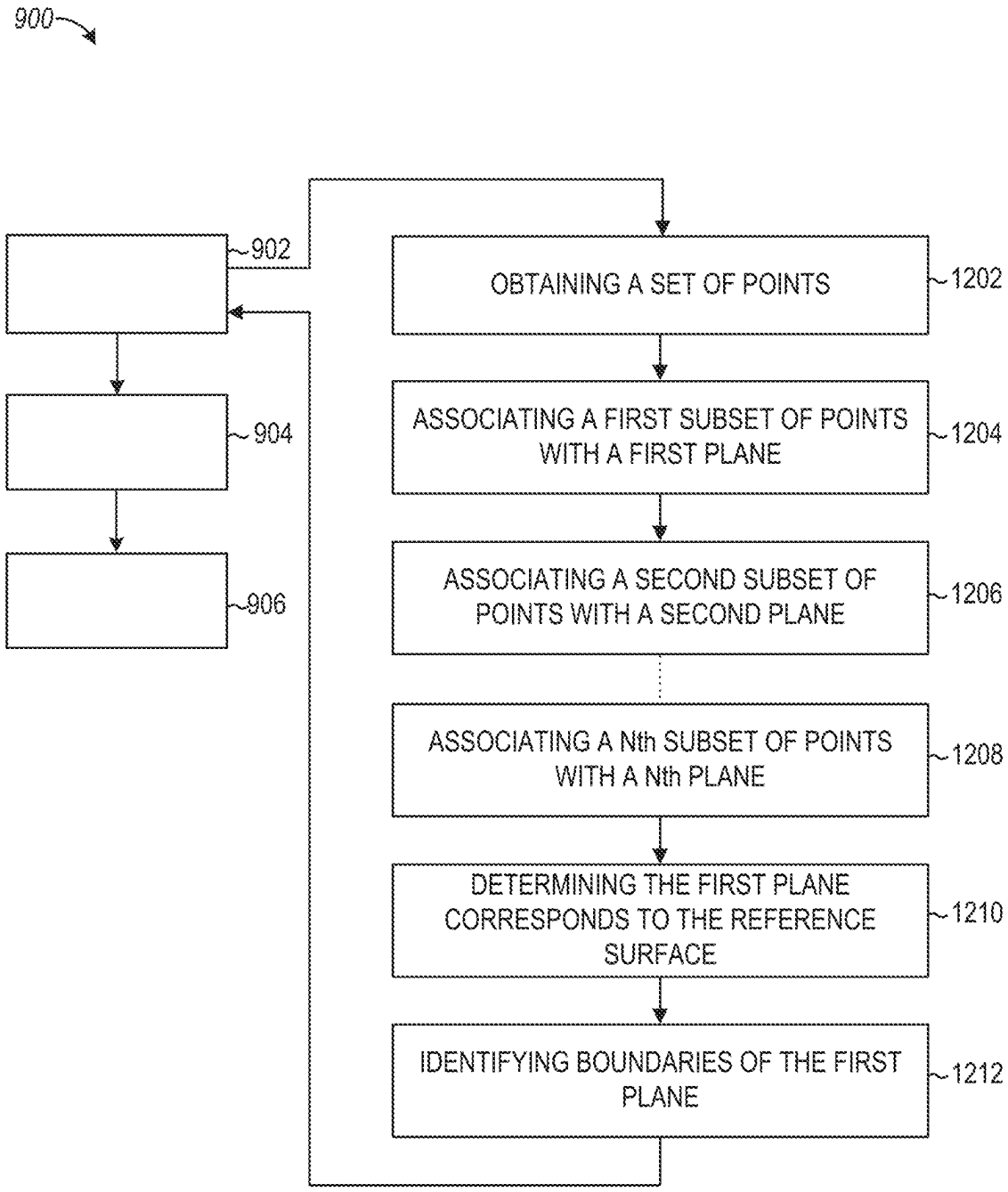

900
902
904
906
OBTAINING A SET OF POINTS
1202
ASSOCIATING A FIRST SUBSET OF POINTS WITH A FIRST PLANE
1204
ASSOCIATING A SECOND SUBSET OF POINTS WITH A SECOND PLANE
1206
ASSOCIATING A Nth SUBSET OF POINTS WITH A Nth PLANE
1208
DETERMINING THE FIRST PLANE CORRESPONDS TO THE REFERENCE SURFACE
1210
IDENTIFYING BOUNDARIES OF THE FIRST PLANE
1212

FIG. 12

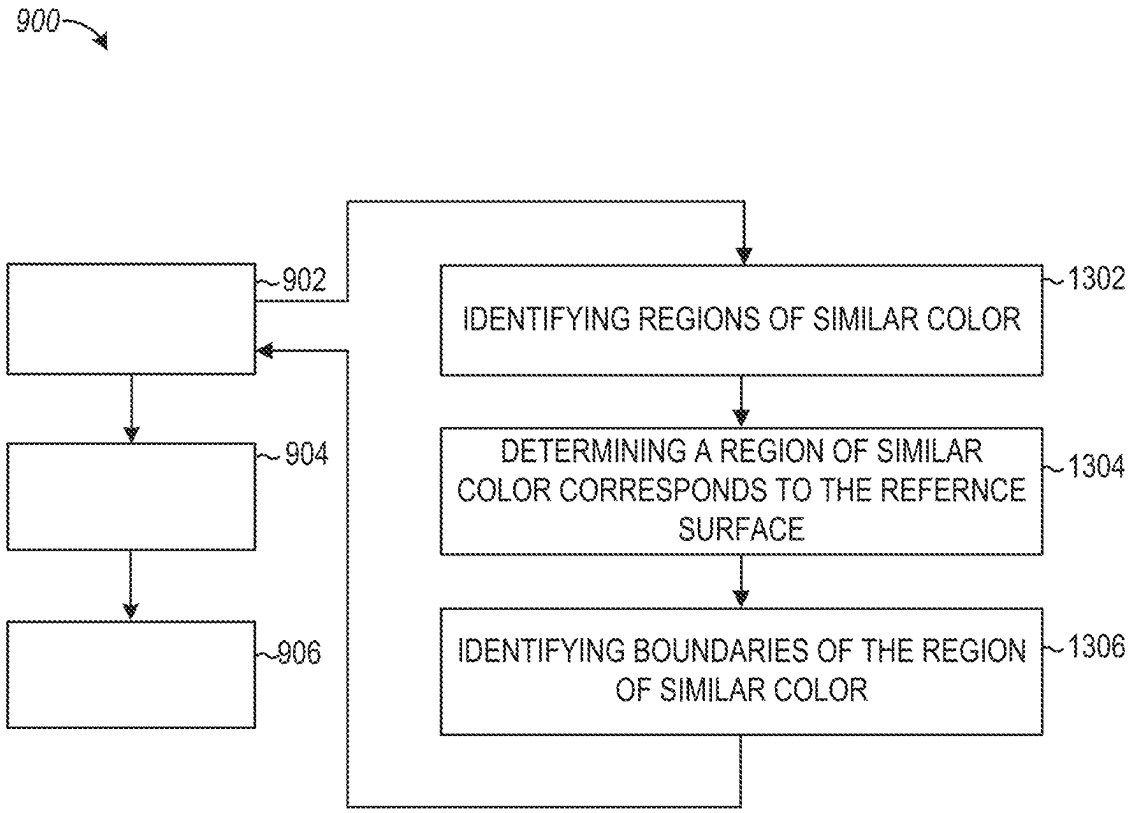

900
902
904
906
IDENTIFYING REGIONS OF SIMILAR COLOR
1302
DETERMINING A REGION OF SIMILAR COLOR CORRESPONDS TO THE REFERNCE SURFACE
1304
IDENTIFYING BOUNDARIES OF THE REGION OF SIMILAR COLOR
1306

FIG. 13

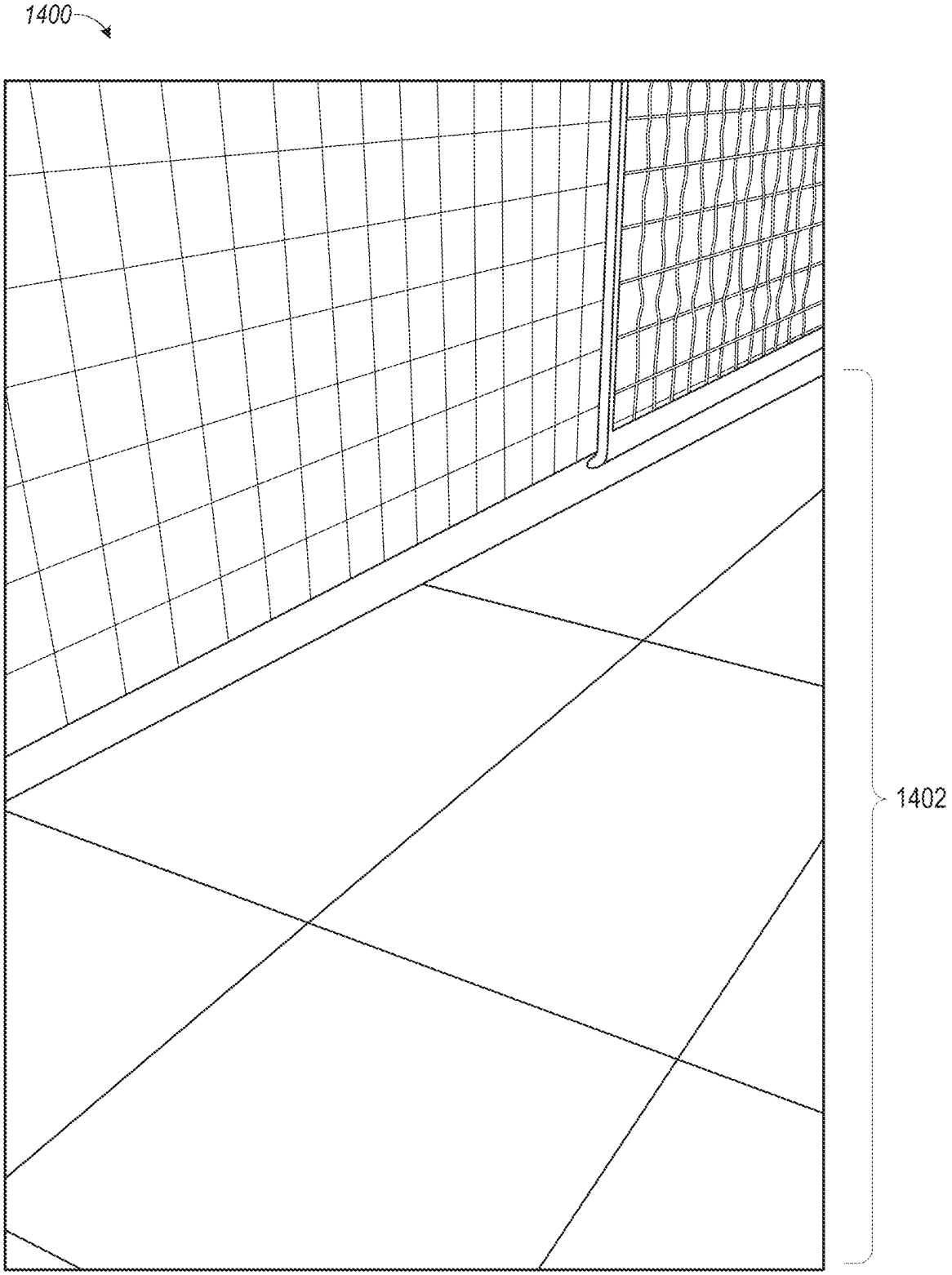


FIG. 14A

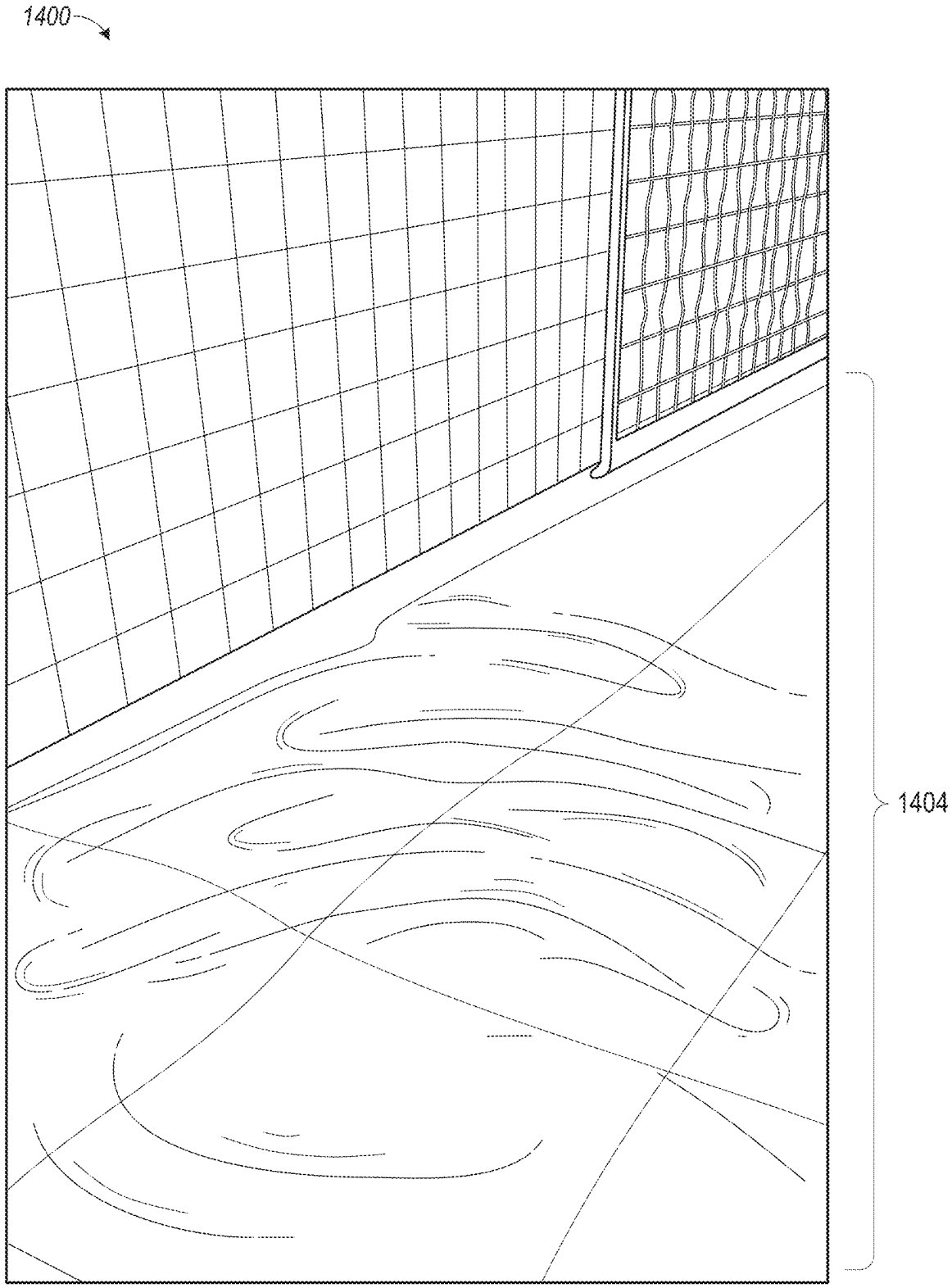


FIG. 14B

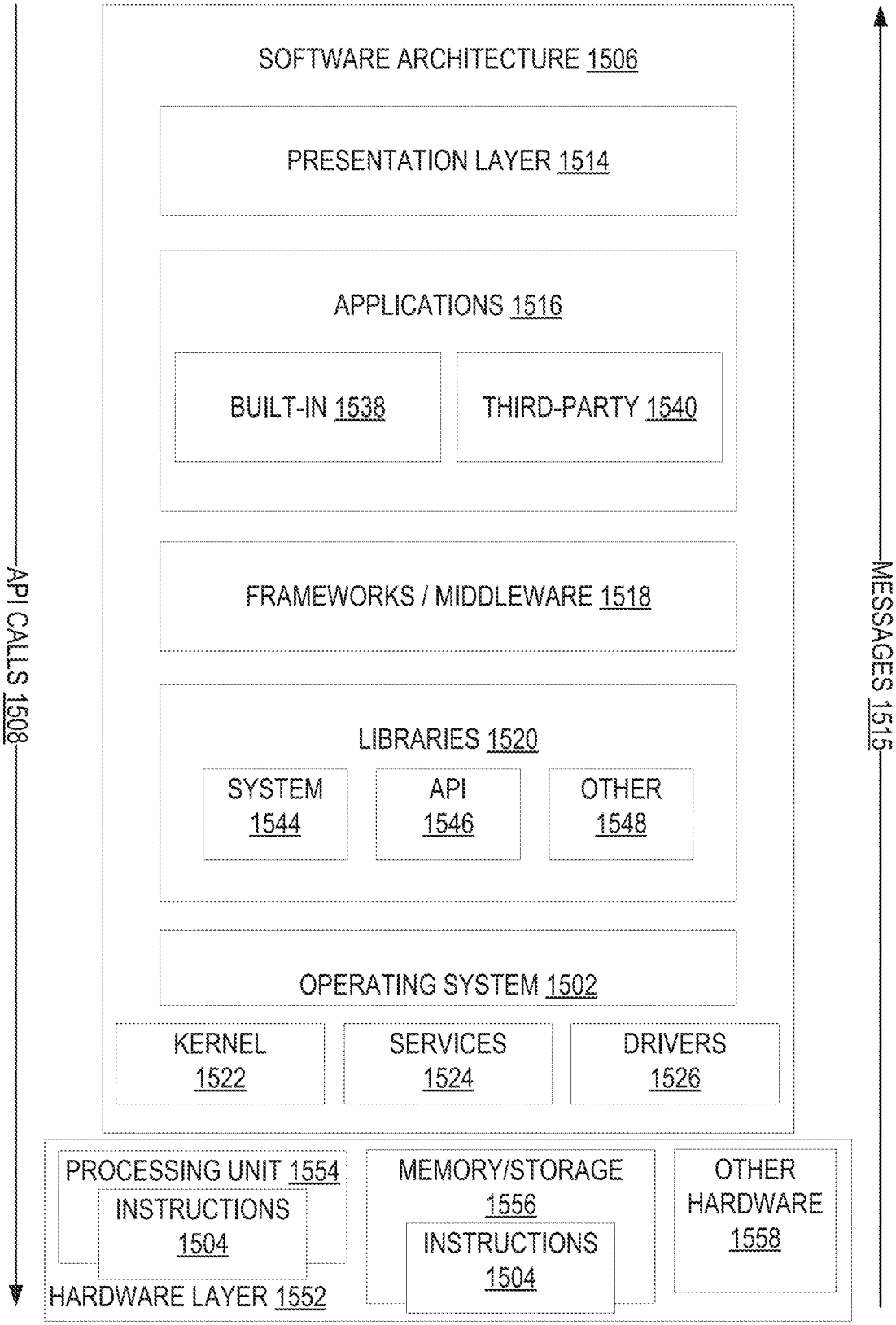

SOFTWARE ARCHITECTURE 1506
PRESENTATION LAYER 1514
APPLICATIONS 1516
BUILT-IN 1538
THIRD-PARTY 1540
FRAMEWORKS / MIDDLEWARE 1518
LIBRARIES 1520
SYSTEM 1544
API 1546
OTHER 1548
OPERATING SYSTEM 1502
KERNEL 1522
SERVICES 1524
DRIVERS 1526
API CALLS 1508
MESSAGES 1515
PROCESSING UNIT 1554
INSTRUCTIONS 1504
MEMORY/STORAGE 1556
INSTRUCTIONS 1504
OTHER HARDWARE 1558
HARDWARE LAYER 1552

FIG. 15

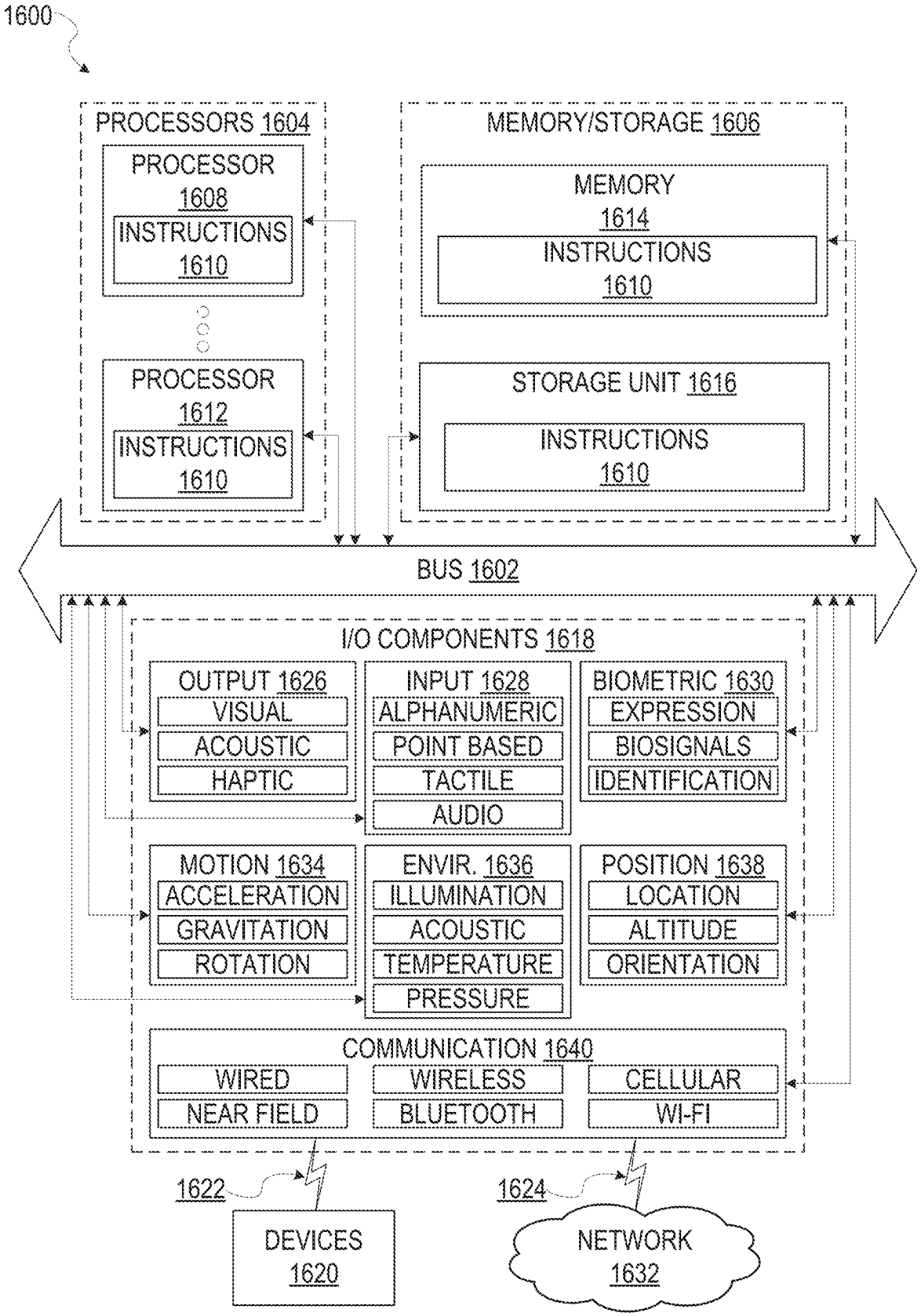

1600
PROCESSORS 1604
PROCESSOR 1608
INSTRUCTIONS 1610
PROCESSOR 1612
INSTRUCTIONS 1610
MEMORY/STORAGE 1606
MEMORY 1614
INSTRUCTIONS 1610
STORAGE UNIT 1616
INSTRUCTIONS 1610
BUS 1602
I/O COMPONENTS 1618
OUTPUT 1626
VISUAL
ACOUSTIC
HAPTIC
INPUT 1628
ALPHANUMERIC
POINT BASED
TACTILE
AUDIO
BIOMETRIC 1630
EXPRESSION
BIOSIGNALS
IDENTIFICATION
MOTION 1634
ACCELERATION
GRAVITATION
ROTATION
ENVIR. 1636
ILLUMINATION
ACOUSTIC
TEMPERATURE
PRESSURE
POSITION 1638
LOCATION
ALTITUDE
ORIENTATION
COMMUNICATION 1640
WIRED
NEAR FIELD
WIRELESS
BLUETOOTH
CELLULAR
WI-FI
1622
DEVICES 1620
1624
NETWORK 1632

FIG. 16

VIRTUAL SURFACE MODIFICATION

PRIORITY CLAIM

[0001] This application is a continuation of U.S. application Ser. No. 17/963,090, filed Oct. 10, 2022, which application is a continuation of U.S. application Ser. No. 16/723,540, filed Dec. 20, 2019, now issued as U.S. Pat. No. 11,501,499, which application is a non-provisional of, and claims the benefit of priority under 35 U.S.C. § 119(e) from, U.S. Provisional Application Ser. No. 62/782,916, entitled "VIRTUAL SURFACE MODIFICATION," filed on Dec. 20, 2018, which are hereby incorporated by reference.

TECHNICAL FIELD

[0002] The present disclosure relates generally to visual presentations and more particularly to rendering virtual modifications to real-world environments depicted in a camera feed.

BACKGROUND

[0003] Virtual rendering systems can be used to create engaging and entertaining augmented reality experiences, in which three-dimensional (3D) virtual object graphics content appears to be present in the real-world. Such systems can be subject to presentation problems due to environmental conditions, user actions, unanticipated visual interruption between a camera and the object being rendered, and the like. This can cause a virtual object to disappear or otherwise behave erratically, which breaks the illusion of the virtual objects being present in the real-world. For example, a virtual rendering system may not present virtual objects in a consistent manner with respect to real-world items as a user moves about through the real-world.

BRIEF DESCRIPTION OF THE DRAWINGS

[0004] In the drawings, which are not necessarily drawn to scale, like numerals may describe similar components in different views. To easily identify the discussion of any particular element or act, the most significant digit or digits in a reference number refer to the figure number in which that element is first introduced. Some embodiments are illustrated by way of example, and not limitation, in the figures of the accompanying drawings in which:

[0005] FIG. **1** is a block diagram showing an example messaging system for exchanging data (e.g., messages and associated content) over a network, according to example embodiments.

[0006] FIG. **2** is block diagram illustrating further details regarding a messaging system, according to example embodiments.

[0007] FIG. **3** is a schematic diagram illustrating data which may be stored in the database of the messaging server system, according to example embodiments.

[0008] FIG. **4** is a schematic diagram illustrating a structure of a message generated by a messaging client application for communication, according to example embodiments.

[0009] FIG. **5** is a schematic diagram illustrating an example access-limiting process, in terms of which access to content (e.g., an ephemeral message-and associated multimedia payload of data) or a content collection (e.g., an ephemeral message story) may be time-limited (e.g., made ephemeral), according to example embodiments.

[0010] FIG. **6** is a block diagram illustrating various components of a virtual rendering system, according to example embodiments.

[0011] FIG. **7** is a flowchart illustrating example operations of the virtual rendering system in performing a method for rendering virtual modification to a 3D space, according to example embodiments.

[0012] FIG. **8** is a flowchart illustrating example operations of the virtual rendering system in performing a method for tracking an object rendered in a 3D space, according to example embodiments.

[0013] FIGS. **9-13** are flowcharts illustrating example operations of the virtual rendering system in performing a method for rendering a virtual modification to a surface in 3D space, according to example embodiments.

[0014] FIGS. **14**A and **14**B are interface diagrams illustrating an interface provided by the virtual rendering system, according to example embodiments.

[0015] FIG. **15** is a block diagram illustrating a representative software architecture, which may be used in conjunction with various hardware architectures herein described, according to example embodiments.

[0016] FIG. **16** is a block diagram illustrating components of a machine able to read instructions from a machine-readable medium (e.g., a machine-readable storage medium) and perform any one or more of the methodologies discussed herein, according to example embodiments.

DETAILED DESCRIPTION

[0017] The description that follows includes systems, methods, techniques, instruction sequences, and computing machine program products that embody illustrative embodiments of the disclosure. In the following description, for the purposes of explanation, numerous specific details are set forth in order to provide an understanding of various embodiments of the inventive subject matter. It will be evident, however, to those skilled in the art, that embodiments of the inventive subject matter may be practiced without these specific details. In general, well-known instruction instances, protocols, structures, and techniques are not necessarily shown in detail.

[0018] Among other things, embodiments of the present disclosure improve the functionality of electronic messaging and imaging software and systems by rendering virtual modifications to 3D real-world environments depicted in image data (e.g., images and video) as if the modifications exist in the real-world environments. For example, the system may render one or more visual effects applied to a real-world surface within a 3D space depicted in image content generated by an image-capturing device (e.g., a digital camera). The one or more visual effects may be rendered such that the modified surface appears to exist in the real-world environment. Visual effects applied to real-world surfaces may be any of a wide range of visual effects including, for example, changing a color of the surface, changing a texture of the surface, applying an animation effect to the surface (e.g., flowing water), blurring the surface, rendering a moving virtual object whose movement is bounded by the boundaries of the surface, replacing the surface with other visual content, and various combinations thereof.

[0019] FIG. **1** is a block diagram showing an example messaging system **100** for exchanging data (e.g., messages and associated content) over a network. The messaging

system **100** includes multiple client devices **102**, each of which hosts a number of applications, including a messaging client application **104**. Each messaging client application **104** is communicatively coupled to other instances of the messaging client application **104** and a messaging server system **108** via a network **106** (e.g., the Internet).

[0020] Accordingly, each messaging client application **104** is able to communicate and exchange data with another messaging client application **104** and with the messaging server system **108** via the network **106**. The data exchanged between messaging client applications **104**, and between a messaging client application **104** and the messaging server system **108**, includes functions (e.g., commands to invoke functions) as well as payload data (e.g., text, audio, video, or other multimedia data).

[0021] The messaging server system **108** provides server-side functionality via the network **106** to a particular messaging client application **104**. While certain functions of the messaging system **100** are described herein as being performed by either a messaging client application **104** or by the messaging server system **108**, it will be appreciated that the location of certain functionality either within the messaging client application **104** or the messaging server system **108** is a design choice. For example, it may be technically preferable to initially deploy certain technology and functionality within the messaging server system **108**, but to later migrate this technology and functionality to the messaging client application **104** where a client device **102** has a sufficient processing capacity.

[0022] The messaging server system **108** supports various services and operations that are provided to the messaging client application **104**. Such operations include transmitting data to, receiving data from, and processing data generated by the messaging client application **104**. This data may include message content, client device information, geolocation information, media annotation and overlays, message content persistence conditions, social network information, and live event information, as examples. Data exchanges within the messaging system **100** are invoked and controlled through functions available via user interfaces (UIs) of the messaging client application **104**.

[0023] Turning now specifically to the messaging server system **108**, an Application Program Interface (API) server **110** is coupled to, and provides a programmatic interface to, an application server **112**. The application server **112** is communicatively coupled to a database server **118**, which facilitates access to a database **120** in which is stored data associated with messages processed by the application server **112**.

[0024] Dealing specifically with the API server **110**, this server receives and transmits message data (e.g., commands and message payloads) between the client device **102** and the application server **112**. Specifically, the API server **110** provides a set of interfaces (e.g., routines and protocols) that can be called or queried by the messaging client application **104** in order to invoke functionality of the application server **112**. The API server **110** exposes various functions supported by the application server **112**, including account registration, login functionality, the sending of messages, via the application server **112**, from a particular messaging client application **104** to another messaging client application **104**, the sending of media files (e.g., images or video) from a messaging client application **104** to the messaging server application **114**, and for possible access by another messaging client application **104**, the setting of a collection of media data (e.g., story), the retrieval of such collections, the retrieval of a list of friends of a user of a client device **102**, the retrieval of messages and content, the adding and deleting of friends to a social graph, the location of friends within a social graph, opening an application event (e.g., relating to the messaging client application **104**).

[0025] The application server **112** hosts a number of applications and subsystems, including a messaging server application **114**, an image processing system **116**, and a social network system **122**. The messaging server application **114** implements a number of message processing technologies and functions, particularly related to the aggregation and other processing of content (e.g., textual and multimedia content) included in messages received from multiple instances of the messaging client application **104**. As will be described in further detail, the text and media content from multiple sources may be aggregated into collections of content (e.g., called stories or galleries). These collections are then made available, by the messaging server application **114**, to the messaging client application **104**. Other processor and memory intensive processing of data may also be performed server-side by the messaging server application **114**, in view of the hardware requirements for such processing.

[0026] As will be discussed below, the messaging server application **114** includes a virtual rendering system that provides functionality to generate, render, and track visual modifications within a 3D real-world environment depicted in a camera view of the client device **102**.

[0027] The application server **112** also includes an image processing system **116** that is dedicated to performing various image processing operations, typically with respect to images or video received within the payload of a message at the messaging server application **114**.

[0028] The social network system **122** supports various social networking functions and services and makes these functions and services available to the messaging server application **114**. To this end, the social network system **122** maintains and accesses an entity graph within the database **120**. Examples of functions and services supported by the social network system **122** include the identification of other users of the messaging system **100** with which a particular user has relationships or is "following," and also the identification of other entities and interests of a particular user.

[0029] The application server **112** is communicatively coupled to a database server **118**, which facilitates access to a database **120** in which is stored data associated with messages processed by the messaging server application **114**.

[0030] FIG. **2** is block diagram illustrating further details regarding the messaging system **100**, according to example embodiments. Specifically, the messaging system **100** is shown to comprise the messaging client application **104** and the application server **112**, which in turn embody a number of subsystems, namely an ephemeral timer system **202**, a collection management system **204**, an annotation system **206**, and a virtual rendering system **210**.

[0031] The ephemeral timer system **202** is responsible for enforcing the temporary access to content permitted by the messaging client application **104** and the messaging server application **114**. To this end, the ephemeral timer system **202** incorporates a number of timers that, based on duration and display parameters associated with a message, or collection

of messages (e.g., a story), selectively display and enable access to messages and associated content via the messaging client application **104**. Further details regarding the operation of the ephemeral timer system **202** are provided below.

[0032] The collection management system **204** is responsible for managing collections of media (e.g., collections of text, image, video, and audio data). In some examples, a collection of content (e.g., messages, including images, video, text, and audio) may be organized into an "event gallery" or an "event story." Such a collection may be made available for a specified time period, such as the duration of an event to which the content relates. For example, content relating to a music concert may be made available as a "story" for the duration of that music concert. The collection management system **204** may also be responsible for publishing an icon that provides notification of the existence of a particular collection to the user interface of the messaging client application **104**.

[0033] The collection management system **204** furthermore includes a curation interface **208** that allows a collection manager to manage and curate a particular collection of content. For example, the curation interface **208** enables an event organizer to curate a collection of content relating to a specific event (e.g., delete inappropriate content or redundant messages). Additionally, the collection management system **204** employs machine vision (or image recognition technology) and content rules to automatically curate a content collection. In certain embodiments, compensation may be paid to a user for inclusion of user generated content into a collection. In such cases, the curation interface **208** operates to automatically make payments to such users for the use of their content.

[0034] The annotation system **206** provides various functions that enable a user to annotate or otherwise modify or edit media content associated with a message. For example, the annotation system **206** provides functions related to the generation and publishing of media overlays for messages processed by the messaging system **100**. The annotation system **206** operatively supplies a media overlay (e.g., a filter) to the messaging client application **104** based on a geolocation of the client device **102**. In another example, the annotation system **206** operatively supplies a media overlay to the messaging client application **104** based on other information, such as social network information of the user of the client device **102**. A media overlay may include audio and visual content and visual effects. Examples of audio and visual content include pictures, texts, logos, animations, and sound effects. An example of a visual effect includes color overlaying. The audio and visual content or the visual effects can be applied to a media content item (e.g., a photo) at the client device **102**. For example, the media overlay includes text that can be overlaid on top of a photograph generated by the client device **102**. In another example, the media overlay includes an identification of a location overlay (e.g., Venice beach), a name of a live event, or a name of a merchant overlay (e.g., Beach Coffee House). In another example, the annotation system **206** uses the geolocation of the client device **102** to identify a media overlay that includes the name of a merchant at the geolocation of the client device **102**. The media overlay may include other indicia associated with the merchant. The media overlays may be stored in the database **120** and accessed through the database server **118**.

[0035] In one example embodiment, the annotation system **206** provides a user-based publication platform that enables users to select a geolocation on a map and upload content associated with the selected geolocation. The user may also specify circumstances under which a particular media overlay should be offered to other users. The annotation system **206** generates a media overlay that includes the uploaded content and associates the uploaded content with the selected geolocation.

[0036] In another example embodiment, the annotation system **206** provides a merchant-based publication platform that enables merchants to select a particular media overlay associated with a geolocation via a bidding process. For example, the annotation system **206** associates the media overlay of a highest bidding merchant with a corresponding geolocation for a predefined amount of time

[0037] The virtual rendering system **210** provides functionality to generate, render, and track virtual modifications within a 3D real-world environment depicted in a live camera feed of the client device **102** (also referred to by those of ordinary skill in the art as a "camera stream," "a video stream," or a "video feed"). The virtual modifications provided by the virtual rendering system **210** may include application of one or more visual effects to real-world surfaces depicted in the camera feed. The virtual modifications provided by the virtual rendering system **210** may also include virtual objects rendered within real-world environments depicted in the live camera feed of the client device **102**.

[0038] FIG. **3** is a schematic diagram **300** illustrating data, which may be stored in the database **120** of the messaging server system **108**, according to certain example embodiments. While the content of the database **120** is shown to comprise a number of tables, it will be appreciated that the data could be stored in other types of data structures (e.g., as an object-oriented database).

[0039] The database **120** includes message data stored within a message table **314**. An entity table **302** stores entity data, including an entity graph **304**. Entities for which records are maintained within the entity table **302** may include individuals, corporate entities, organizations, objects, places, events, and so forth. Regardless of type, any entity regarding which the messaging server system **108** stores data may be a recognized entity. Each entity is provided with a unique identifier, as well as an entity type identifier (not shown).

[0040] The entity graph **304** furthermore stores information regarding relationships and associations between entities. Such relationships may be social, professional (e.g., work at a common corporation or organization), interested-based, or activity-based, merely for example.

[0041] The database **120** also stores annotation data, in the example form of filters and lenses, in an annotation table **312**. Filters and lens for which data is stored within the annotation table **312** are associated with and applied to videos (for which data is stored in a video table **310**) and/or images (for which data is stored in an image table **308**). Filters are overlays that are displayed as overlaid on an image or video during presentation to a recipient user. Lenses include real-time visual effects and/or sounds that may be added to real-world environments depicted in a camera feed (e.g., while a user is viewing the camera feed via one or more interfaces of the messaging client application **104**, while composing a message, or during presentation to a recipient user). In some embodiments, filters are applied to an image or video after the image or video is captured at

the client device **102** while a lens is applied to the camera feed of the client device **102** such that when an image or videos is captured at the client device **102** with a lens applied, the applied lens is incorporated as part of the image or video that is generated. Filters and lenses may be of various types, including user-selected filters and lens from a gallery of filters or a gallery of lenses presented to a sending user by the messaging client application **104** when the sending user is composing a message.

[0042] As mentioned above, the video table **310** stores video data which, in one embodiment, is associated with messages for which records are maintained within the message table **314**. Similarly, the image table **308** stores image data associated with messages for which message data is stored in the entity table **302**. The entity table **302** may associate various annotations from the annotation table **312** with various images and videos stored in the image table **308** and the video table **310**.

[0043] A story table **306** stores data regarding collections of messages and associated image, video, or audio data, which are compiled into a collection (e.g., a story or a gallery). The creation of a particular collection may be initiated by a particular user (e.g., each user for which a record is maintained in the entity table **302**). A user may create a "personal story" in the form of a collection of content that has been created and sent/broadcast by that user. To this end, the UI of the messaging client application **104** may include an icon that is user selectable to enable a sending user to add specific content to his or her personal story.

[0044] A collection may also constitute a "live story," which is a collection of content from multiple users that is created manually, automatically, or using a combination of manual and automatic techniques. For example, a "live story" may constitute a curated stream of user-submitted content from various locations and events. Users, whose client devices have location services enabled and are at a common location event at a particular time, may, for example, be presented with an option, via a user interface of the messaging client application **104**, to contribute content to a particular live story. The live story may be identified to the user by the messaging client application **104**, based on his or her location. The end result is a "live story" told from a community perspective.

[0045] A further type of content collection is known as a "location story," which enables a user whose client device **102** is located within a specific geographic location (e.g., on a college or university campus) to contribute to a particular collection. In some embodiments, a contribution to a location story may require a second degree of authentication to verify that the end user belongs to a specific organization or other entity (e.g., is a student on the university campus).

[0046] FIG. **4** is a schematic diagram illustrating a structure of a message **400**, according to some embodiments, generated by a messaging client application **104** for communication to a further messaging client application **104** or the messaging server application **114**. The content of a particular message **400** is used to populate the message table **314** stored within the database **120**, accessible by the messaging server application **114**. Similarly, the content of a message **400** is stored in memory as "in-transit" or "in-flight" data of the client device **102** or the application server **112**. The message **400** is shown to include the following components:

[0047] A message identifier **402**: a unique identifier that identifies the message **400**.

[0048] A message text payload **404**: text, to be generated by a user via a user interface of the client device **102** and that is included in the message **400**.

[0049] A message image payload **406**: image data, captured by a camera component of a client device **102** or retrieved from memory of a client device **102**, and that is included in the message **400**.

[0050] A message video payload **408**: video data, captured by a camera component or retrieved from a memory component of the client device **102** and that is included in the message **400**.

[0051] A message audio payload **410**: audio data, captured by a microphone or retrieved from the memory component of the client device **102**, and that is included in the message **400**.

[0052] A message annotations **412**: annotation data (e.g., filters, stickers or other enhancements) that represents annotations to be applied to message image payload **406**, message video payload **408**, or message audio payload **410** of the message **400**.

[0053] A message duration parameter **414**: parameter value indicating, in seconds, the amount of time for which content of the message (e.g., the message image payload **406**, message video payload **408**, message audio payload **410**) is to be presented or made accessible to a user via the messaging client application **104**.

[0054] A message geolocation parameter **416**: geolocation data (e.g., latitudinal and longitudinal coordinates) associated with the content payload of the message. Multiple message geolocation parameter **416** values may be included in the payload, with each of these parameter values being associated with respect to content items included in the content (e.g., a specific image into within the message image payload **406**, or a specific video in the message video payload **408**).

[0055] A message story identifier **418**: identifier value identifying one or more content collections (e.g., "stories") with which a particular content item in the message image payload **406** of the message **400** is associated. For example, multiple images within the message image payload **406** may each be associated with multiple content collections using identifier values.

[0056] A message tag **420**: each message **400** may be tagged with multiple tags, each of which is indicative of the subject matter of content included in the message payload. For example, where a particular image included in the message image payload **406** depicts an animal (e.g., a lion), a tag value may be included within the message tag **420** that is indicative of the relevant animal. Tag values may be generated manually, based on user input, or may be automatically generated using, for example, image recognition.

[0057] A message sender identifier **422**: an identifier (e.g., a messaging system identifier, email address, or device identifier) indicative of a user of the client device **102** on which the message **400** was generated and from which the message **400** was sent.

[0058] A message receiver identifier **424**: an identifier (e.g., a messaging system identifier, email address or device identifier) indicative of a user of the client device **102** to which the message **400** is addressed.

[0059] The contents (e.g. values) of the various components of message **400** may be pointers to locations in tables within which content data values are stored. For example, an image value in the message image payload **406** may be a pointer to (or address of) a location within an image table **308**. Similarly, values within the message video payload **408** may point to data stored within a video table **310**, values stored within the message annotations **412** may point to data stored in an annotation table **312**, values stored within the message story identifier **418** may point to data stored in a story table **306**, and values stored within the message sender identifier **422** and the message receiver identifier **424** may point to user records stored within an entity table **302**.

[0060] FIG. **5** is a schematic diagram illustrating an access-limiting process **500**, in terms of which access to content (e.g., an ephemeral message **502**, and associated multimedia payload of data) or a content collection (e.g., an ephemeral message story **504**) may be time-limited (e.g., made ephemeral).

[0061] An ephemeral message **502** is shown to be associated with a message duration parameter **506**, the value of which determines an amount of time that the ephemeral message **502** will be displayed to a receiving user of the ephemeral message **502** by the messaging client application **104**. In one embodiment, where the messaging client application **104** is an application client, an ephemeral message **502** is viewable by a receiving user for up to a maximum of 10 seconds, depending on the amount of time that the sending user specifies using the message duration parameter **506**.

[0062] The message duration parameter **506** and the message receiver identifier **424** are shown to be inputs to a message timer **512**, which is responsible for determining the amount of time that the ephemeral message **502** is shown to a particular receiving user identified by the message receiver identifier **424**. In particular, the ephemeral message **502** will only be shown to the relevant receiving user for a time period determined by the value of the message duration parameter **506**. The message timer **512** is shown to provide output to a more generalized ephemeral timer system **202**, which is responsible for the overall timing of display of content (e.g., an ephemeral message **502**) to a receiving user.

[0063] The ephemeral message **502** is shown in FIG. **5** to be included within an ephemeral message story **504** (e.g., a personal story, or an event story). The ephemeral message story **504** has an associated story duration parameter **508**, a value of which determines a time-duration for which the ephemeral message story **504** is presented and accessible to users of the messaging system **100**. The story duration parameter **508**, for example, may be the duration of a music concert, where the ephemeral message story **504** is a collection of content pertaining to that concert. Alternatively, a user (either the owning user or a curator user) may specify the value for the story duration parameter **508** when performing the setup and creation of the ephemeral message story **504**.

[0064] Additionally, each ephemeral message **502** within the ephemeral message story **504** has an associated story participation parameter **510**, a value of which determines the duration of time for which the ephemeral message **502** will be accessible within the context of the ephemeral message story **504**. Accordingly, a particular ephemeral message story **504** may "expire" and become inaccessible within the context of the ephemeral message story **504**, prior to the ephemeral message story **504** itself expiring in terms of the story duration parameter **508**. The story duration parameter **508**, story participation parameter **510**, and message receiver identifier **424** each provide input to a story timer **514**, which operationally determines, firstly, whether a particular ephemeral message **502** of the ephemeral message story **504** will be displayed to a particular receiving user and, if so, for how long. Note that the ephemeral message story **504** is also aware of the identity of the particular receiving user as a result of the message receiver identifier **424**.

[0065] Accordingly, the story timer **514** operationally controls the overall lifespan of an associated ephemeral message story **504**, as well as an individual ephemeral message **502** included in the ephemeral message story **504**. In one embodiment, each and every ephemeral message **502** within the ephemeral message story **504** remains viewable and accessible for a time-period specified by the story duration parameter **508**. In a further embodiment, a certain ephemeral message **502** may expire, within the context of ephemeral message story **504**, based on a story participation parameter **510**. Note that a message duration parameter **506** may still determine the duration of time for which a particular ephemeral message **502** is displayed to a receiving user, even within the context of the ephemeral message story **504**. Accordingly, the message duration parameter **506** determines the duration of time that a particular ephemeral message **502** is displayed to a receiving user, regardless of whether the receiving user is viewing that ephemeral message **502** inside or outside the context of an ephemeral message story **504**.

[0066] The ephemeral timer system **202** may furthermore operationally remove a particular ephemeral message **502** from the ephemeral message story **504** based on a determination that it has exceeded an associated story participation parameter **510**. For example, when a sending user has established a story participation parameter **510** of 24 hours from posting, the ephemeral timer system **202** will remove the relevant ephemeral message **502** from the ephemeral message story **504** after the specified 24 hours. The ephemeral timer system **202** also operates to remove an ephemeral message story **504** either when the story participation parameter **510** for each and every ephemeral message **502** within the ephemeral message story **504** has expired, or when the ephemeral message story **504** itself has expired in terms of the story duration parameter **508**.

[0067] In certain use cases, a creator of a particular ephemeral message story **504** may specify an indefinite story duration parameter **508**. In this case, the expiration of the story participation parameter **510** for the last remaining ephemeral message **502** within the ephemeral message story **504** will determine when the ephemeral message story **504** itself expires. In this case, a new ephemeral message **502**, added to the ephemeral message story **504**, with a new story participation parameter **510**, effectively extends the life of an ephemeral message story **504** to equal the value of the story participation parameter **510**.

[0068] Responsive to the ephemeral timer system **202** determining that an ephemeral message story **504** has expired (e.g., is no longer accessible), the ephemeral timer system **202** communicates with the messaging system **100** (and, for example, specifically the messaging client application **104**) to cause an indicium (e.g., an icon) associated with the relevant ephemeral message story **504** to no longer be displayed within a user interface of the messaging client

application **104**. Similarly, when the ephemeral timer system **202** determines that the message duration parameter **506** for a particular ephemeral message **502** has expired, the ephemeral timer system **202** causes the messaging client application **104** to no longer display an indicium (e.g., an icon or textual identification) associated with the ephemeral message **502**.

[0069] FIG. **6** is a block diagram illustrating functional components of the virtual rendering system **210** that configure the virtual rendering system **210** to render virtual modifications to a 3D real-world environment depicted in a live camera feed. For example, the virtual rendering system **210** may render virtual modifications to real-world surfaces in a 3D space depicted in a live camera feed. As another example, the virtual rendering system **210** may render virtual objects within the 3D space.

[0070] The virtual rendering system **210** is shown as including a rendering component **602**, a tracking system **604**, a disruption detection component **606**, an object template component **608**, and an event detection component **610**. The various components of the virtual rendering system **210** may be configured to communicate with each other (e.g., via a bus, shared memory, or a switch). Although not illustrated in FIG. **6**, in some embodiments, the virtual rendering system **210** may include or may be in communication with a camera configured to produce a camera feed comprising image data that includes a sequence of images (e.g., a video).

[0071] Any one or more of the components described may be implemented using hardware alone (e.g., one or more of the processors **612** of a machine) or a combination of hardware and software. For example, any component described of the virtual rendering system **210** may physically include an arrangement of one or more of the processors **612** (e.g., a subset of or among the one or more processors of the machine) configured to perform the operations described herein for that component. As another example, any component of the virtual rendering system **210** may include software, hardware, or both, that configure an arrangement of one or more processors **612** (e.g., among the one or more processors of the machine) to perform the operations described herein for that component. Accordingly, different components of the virtual rendering system **210** may include and configure different arrangements of such processors **612** or a single arrangement of such processors **612** at different points in time.

[0072] Moreover, any two or more components of the virtual rendering system **210** may be combined into a single component, and the functions described herein for a single component may be subdivided among multiple components. Furthermore, according to various example embodiments, components described herein as being implemented within a single machine, database, or device may be distributed across multiple machines, databases, or devices.

[0073] The tracking system **604** may comprise a first tracking sub-system **604**A, a second tracking sub-system **604**B, and a third tracking sub-system **604**C. Each tracking sub-system tracks the position of a virtual modification to a 3D space based on a set of tracking indicia.

[0074] Tracking systems are subject to frequent tracking failure due to environmental conditions, user actions, unanticipated visual interruption between camera and object/scene being tracked, and so forth. Traditionally, such tracking failures would cause a disruption in the presentation of virtual objects in a 3D space. For example, a virtual object may disappear or otherwise behave erratically, thereby interrupting the illusion of the virtual object being presented within the 3D space. This undermines the perceived quality of the 3D experience as a whole.

[0075] Traditional tracking systems rely on a single approach (Natural Feature Tracking (NFT), Simultaneous Localization And Mapping (SLAM), Gyroscopic, etc.) that each have breaking points in real-world usage due to inaccurate sensor data, movement, loss or occlusion of visual marker, or dynamic interruptions to a scene. Further, each approach may have individual limitations in capability. For example, a gyroscopic tracking system can only track items with three degrees of freedom (3 DoF). Further, utilization of a single tracking system provides inaccurate or unstable position estimation, due to inherent limitations of each individual system. For example, an NFT system may not provide sufficient pitch, yaw, or roll estimation due to the inaccuracies of visual tracking alone, while gyroscopic tracking systems provide inaccurate translation (up, down, left, right).

[0076] To address the foregoing issues with traditional tracking systems, the virtual rendering system **210** comprises multiple redundant tracking sub-systems **604**A-C that enable seamless transitions between tracking sub-systems. The multiple redundant tracking sub-systems **604**A-C address the issues with traditional tracking systems by merging multiple tracking approaches into a single tracking system **604**. The tracking system **604** is able to combine 6 DoF and 3 DoF tracking techniques through combining and transitioning between multiple tracking systems based on the availability of tracking indicia tracked by the tracking systems. Thus, as the indicia tracked by any one tracking system becomes unavailable, the virtual rendering system **210** seamlessly switches between tracking in 6 DoF and 3 DoF, thereby providing the user with an uninterrupted experience. For example, in the case of visual tracking systems (e.g., NFT, SLAM), tracking indicia typically analyzed to determine orientation may be replaced with gyroscopic tracking indicia from a gyroscopic tracking system. This would thereby enable transitioning between tracking in 6 Dof and 3 DoF based on the availability of tracking indicia.

[0077] In some example embodiments, to transition between tracking in 6 DoF and 3 DoF, the virtual rendering system **210** gathers and stores tracking indicia within a tracking matrix that includes translation indicia (e.g., up, down, left, right) and rotation indicia (e.g., pitch, yaw, roll). The translation indicia gathered by an NFT system may thereby be extracted from the tracking matrix and utilized when future translation indicia gathered by the NFT system become inaccurate or unavailable. In the meantime, the rotation indicia continue to be provided by the gyroscope. In this way, when the mobile device loses tracking indicia, the tracked objects that are presented in the 3D space will not be changed abruptly at the frame when the tracking indicia are lost. Subsequently, when the target tracking object reappears in the screen, and a new translation T_1 is obtained, the translation part of the view matrix will then be taking advantage of the new translation $T_1$and use T_1-T_0 as the translation of the view matrix.

[0078] The rendering component **602** of the virtual rendering system **210** is configured to render virtual modifications in a 3D space captured within a live camera feed

produced by a camera of the client device **102**. For example, the rendering component **602** may render a visual effect applied to real-world surface in a 3D space captured within the live camera feed. In rendering the modification, the virtual rendering system **210** dynamically applies an image mask to the surface depicted in the live camera feed and applies the visual effect to the image mask.

[0079] The virtual rendering system **210** may track and adjust the position of a virtual modification by one or more tracking systems in 6 DoF. For example, the one or more tracking systems of the virtual rendering system **210** may collect and analyze a set of tracking indicia (e.g., roll, pitch, yaw, natural features, etc.) in order to track the position of the virtual modification relative to the client device **102** in the 3D space with 6 DoF. In such embodiments, the virtual rendering system **210** may transition between tracking systems based on the availability of the tracked indicia to maintain consistent tracking in 6 DoF.

[0080] The disruption detection component **606** monitors tracking indicia to detect disruptions. Upon the disruption detection component **606** detecting an interruption of one or more indicia, such that tracking in 6 DoF becomes unreliable or impossible, the virtual rendering system **210** transitions to tracking the virtual modification in the 3D space in 3 DoF in order to prevent an interruption of the display. For example, the virtual rendering system **210** may transition from a first tracking system (or first set of tracking systems among the set of tracking systems) to a second tracking system among the set of tracking systems (or second set of tracking systems), wherein the second tracking system is capable of tracking the virtual modification with 3 DoF in the 3D space, based on the tracking indicia available.

[0081] In some example embodiments, the set of tracking systems of the virtual rendering system **210** includes a gyroscopic tracking system, an NFT system, and a SLAM tracking system. Each tracking system among the set of tracking systems may analyze tracking indicia in order to track a position of a virtual object within a 3D space. For example, to track a virtual object with 6 DoF, the virtual rendering system **210** may require at least six tracking indicia to be available. As tracking indicia become obstructed or unavailable for various reasons, the virtual rendering system **210** may transition between the available tracking systems among the set of tracking systems in order to maintain 6 DoF or transition to 3 DoF, if necessary.

[0082] It will be readily appreciated that the virtual rendering system **210** provides consistent rendered virtual modifications (e.g., visual effects applied to real-world surface) in real-world 3D spaces in a wide variety of environments and situations. In many applications it can be desirable to provide firm consistency for the locations of these virtual modifications as one or more users, cameras, or other tracking items move around in the environment. This can involve the recognition and use of a specific fixed reference point (e.g., a fixed surface) in the real-world environment. Not using a fixed reference point or item can result in floating or other undesirable inconsistencies in the rendering and presentation of the virtual objects.

[0083] To ensure firm consistency in the location of virtual objects, annotation data in the example form of a presentation lens that is specific for virtual modification tracking and rendering described herein may be employed. In particular, a surface aware lens is a presentation lens that identifies and references a real-world surface (e.g., the ground) for the consistent rendering and presentation of virtual modifications in 3D space. The surface aware lens can be a specific portion or subcomponent within the rendering component **602**. This surface aware lens of the rendering component **602** can be configured to recognize a reference surface based on visual camera content, and may also utilize other device inputs (e.g., gyroscope, accelerometer, compass) to determine what is an appropriate surface within a 3D space depicted in a live camera feed. Once the reference surface has been determined, then virtual modifications can be accomplished with respect to that reference surface. In an example, the reference surface in the 3D space is a ground surface. The virtual rendering system **210** may modify the ground surface as depicted in a live camera feed by applying a visual effect to the ground surface. The virtual rendering system **210** may also render a virtual object at a position in the 3D space such that the caption appears to be anchored to the ground surface.

[0084] In some embodiments, the virtual rendering system **210** may render a virtual modification to a 3D space depicted in a live camera feed of the client device **102** in response to a triggering event. To this end, the event detection component **610** is responsible for detecting such triggering events. The event detection component **610** may detect a triggering event based on data received from one or more components of the client device **102** or from one or more external sources accessible via the network **106**. For example, the triggering event may be based on geolocation data from a location component of the client device **102**, and the detecting of the triggering event may include detecting the client device **102** being at or near a particular geographic location. As another example, the triggering event may be based on a temporal factor and the detecting of the triggering event may include detecting a particular date or time based on a clock signal maintained by the client device **102**. As yet another example, the triggering event may be based on weather data (e.g., obtained from an external source over the network **106**) that describes weather conditions, and the detecting of the triggering event may include detecting a certain weather condition (e.g., snow, rain, wind, etc.).

[0085] FIG. **7** is a flowchart illustrating a method **700** for rendering a virtual modification in a 3D space, according to various embodiments of the present disclosure. The method **700** may be embodied in computer-readable instructions for execution by one or more processors such that the operations of the method **700** may be performed in part or in whole by the functional components of the virtual rendering system **210**; accordingly, the method **700** is described below by way of example with reference thereto. However, it shall be appreciated that at least some of the operations of the method **700** may be deployed on various other hardware configurations and the method **700** is not intended to be limited to the virtual rendering system **210**.

[0086] As depicted in operation **702**, the virtual rendering system **210** receives an input to activate a surface aware lens. This input can be in the form of a manual user input, which can be, for example, a button tap or holding or pointing an active camera in such a manner so as to indicate selection of the surface aware lens. The surface aware lens may, for example, be used with any of the virtual objects for which a template is maintained by the object template component **608**, although the surface aware lens is not limited in application to the virtual object templates maintained by the object template component **608**.

[0087] At operation **704**, the rendering component **602** responds to the input by detecting a real-world reference surface in 3D space depicted in a live camera feed produced by the camera. The camera feed comprises image data that includes a sequence of images (e.g., video) in which the 3D space is depicted. In some embodiments, the reference surface can be a user specified reference surface. As such, the detecting of the reference surface is based on user input such as a tap or other gesture used to activate the surface lens to indicate a reference surface. Such a reference surface can be the floor surface or the ground surface in many cases, although other fixed and ascertainable surfaces can also be used. For example, the rendering component **602** may determine the reference surface by identifying a fixed surface based on an analysis of visual camera content and may also utilize other device inputs (e.g., gyroscope, accelerometer, compass) to ascertain what is an appropriate surface within a 3D space captured by the camera feed. In various embodiments, a confirmation that the proper reference surface has been indicated or highlighted can be requested from the user. In some situations, the system may indicate that a proper reference surface cannot be detected, such that further input or help from the user may be needed.

[0088] At operation **706**, the rendering component **602** orients a virtual modification based on the detected reference surface. The orienting of the virtual modification may include assigning the virtual modification, such as a virtual object, to a position in 3D space based on the detected reference surface and identifying tracking indicia to be used by the tracking system **604** in tracking the virtual object in the 3D space. The position to which the virtual modification is assigned may correspond to the reference surface or a predefined distance above the reference surface. One or both of operations **704** and **706** can also be referred to as initialization of the rendering component **602**. In essence, the determined reference surface within the camera feed is being established in the rendering component **602** at a proper static orientation relative to the reference surface in the real-world.

[0089] At operation **708**, the rendering component **602** renders the virtual modification with respect to the reference surface. For example, the rendering component **602** may render a virtual object with respect to the reference surface. The rendering of the virtual object with respect to the reference surface may include rendering and maintaining the virtual object at the assigned position within the 3D space. Thus, in instances in which the assigned position is a predefined distance from the reference surface, the rendering of the virtual object may include rendering and maintaining the virtual object at the predefined distance from the reference surface. In these instances, the virtual object, when rendered, may not actually contact or rest against the reference surface, but rather may be hovering above or extending away from the reference surface at the predefined distance.

[0090] FIG. **8** is a flowchart illustrating operations of the virtual rendering system **210** in performing a method **800** for tracking virtual modification at a position relative to the client device **102** in a 3D space, according to certain example embodiments. The method **800** may be embodied in computer-readable instructions for execution by one or more processors such that the operations of the method **800** may be performed in part or in whole by the functional components of the virtual rendering system **210**; accordingly, the method **800** is described below by way of example with reference thereto. However, it shall be appreciated that at least some of the operations of the method **800** may be deployed on various other hardware configurations and the method **800** is not intended to be limited to the virtual rendering system **210**.

[0091] At operation **802**, the rendering component **602** renders a virtual modification to a 3D space depicted in a camera feed at a position relative to the client device **102** in a 3D space.

[0092] At operation **804**, the tracking system **604** tracks the virtual modification in 6DoF at the position in the 3D space via the first tracking sub-system **604**A, or a combination of multiple tracking sub-systems (e.g., the first tracking sub-system **604**A and the second tracking sub-system **604**B), based on a set of tracking indicia. When tracking the virtual modification in 6 DoF, a user viewing the modification on the client device **102** can turn or move in any direction without disrupting tracking of the modification. For example, the tracking system **604** may track the position of the virtual modification based on a combination of an NFT system and a gyroscopic tracking system.

[0093] At operation **806**, the disruption detection component **606** detects an interruption of a tracking indicia from among the tracking indicia tracked by the tracking sub-systems (e.g., the first tracking sub-system **604**A). For example, the first tracking sub-system **604**A may include a NFT system configured to rely on tracking indicia that include features of an environment or active light sources in proximity to virtual modifications within the environment (e.g., the ground's plane, or the horizon). The NFT system of the first tracking sub-system **604**A may therefore rely on the positions of three or more known features in the environment to determine the position of the virtual modifications relative to the client device **102** in the 3D space. Should any one or more of the tracking indicia tracked by the first tracking sub-system **604**A become obstructed or unavailable, the tracking of the virtual modification in the 3D space would become disrupted.

[0094] At operation **808**, in response to the disruption detection component **606** detecting a disruption of one or more tracking indicia, the tracking system **604** transitions to one or more other tracking sub-systems (e.g., the second tracking sub-system **604**B and/or the third tracking sub-system **604**C) to maintain tracking of the virtual object relative to the client device **102** in the 3D space. In doing so, the virtual rendering system **210** may transition from 6 DoF to 3 DoF, wherein 3 DoF measures pitch, roll, and yaw, but does not measure translations. As the tracking indicia again become available, the virtual rendering system **210** may thereby transition from 3 DoF back to 6 DoF. For example, when the NFT system becomes unavailable, the tracking system **604** may utilize the last tracking indicia gathered and tracked by the NFT system throughout the subsequent 3 DoF experience.

[0095] FIGS. **9-13** are flowcharts illustrating example operations of the virtual rendering system in performing a method **900** for rendering a virtual modification to a surface in 3D space, according to example embodiments. The method **900** may be embodied in computer-readable instructions for execution by one or more processors such that the operations of the method **900** may be performed in part or in whole by the functional components of the virtual rendering system **210**; accordingly, the method **900** is described below by way of example with reference thereto. However,

it shall be appreciated that at least some of the operations of the method **900** may be deployed on various other hardware configurations and the method **900** is not intended to be limited to the virtual rendering system **210**.

[0096] At operation **902**, the rendering component **602** detects a reference surface in a 3D space depicted in a camera feed produced by a camera of a computing device (e.g., the client device **102**). The camera feed comprises a sequence of images, with each image depicting the 3D space. As previously noted, the reference surface may be the ground surface, although any other fixed and ascertainable surfaces may also be used. For example, the rendering component **602** may detect the reference surface by identifying a fixed surface based on an analysis of visual camera content, and may also utilize other device inputs (e.g., gyroscope, accelerometer, compass) to ascertain what is an appropriate surface within the 3D space depicted in the camera feed.

[0097] In some embodiments, the detecting of the reference surface may be based on user input received on a presentation of the camera feed. This input can be in the form of a manual user input, which can be, for example, a button tap or holding or pointing an active camera in such a manner so as to indicate that a surface is being referenced. In other embodiments, which will be discussed below in reference to FIG. **10**, the detecting of the reference surface may be in response to detecting a triggering event associated with the reference surface.

[0098] At operation **904**, the rendering component **602** dynamically applies an image mask to the reference surface within the 3D space depicted within the camera feed. More specifically, the rendering component **602** applies an image mask to each of the multiple images of the camera feed at the location of the reference surface depicted in each image, which may vary due to changes in the orientation or distance of the camera relative to the reference surface. In general, the applying of the image mask to each image includes classifying pixels in the image based on whether they are inside or outside of the boundaries of the reference surface. Such a pixel classification process may be based on a determined geometry of the 3D space, determined regions of color within an image, or determined regions of photometric consistency within an image.

[0099] In some embodiments, the applying of the image mask may comprise applying an image segmentation neural network to the multiple images of the camera feed. The image segmentation neural network may be trained to perform image segmentation on the images to partition each image into multiple image segments (e.g., sets of pixels). More specifically, the image segmentation neural network may be trained to assign a first label to pixels that are inside the boundaries of the reference surface and assign a second label to pixels that are outside the boundaries of the reference surface. The applying of the image segmentation neural network to an image of the camera feed yields a segmented image with two image segments—a first image segment that includes a first set of pixels assigned to the first label corresponding to the reference surface; and a second image segment that includes a second set of pixels assigned to the second label corresponding to the remainder of the 3D space depicted in the image.

[0100] At operation **906**, the rendering component **602** applies a visual effect to the image mask corresponding to the reference surface. The applying of the visual effect to the image mask causes a modified surface to be rendered in a presentation of the camera feed on a display of the computing device. The visual effect may be any of a wide range of visual effects including, for example, changing a color of the surface, changing a texture of the surface, applying an animation effect to the surface (e.g., flowing water), blurring the surface, rendering a moving virtual object whose movement is bounded by the boundaries of the surface, replacing the surface with other visual content, and various combinations thereof. An example application of a visual effect to a reference surface is illustrated in FIGS. **14**A and **14**B and described below.

[0101] As shown in FIG. **10**, the method **900** may, in some embodiments, also include operations **901** and **905**. The operation **901** may be performed prior to operation **902**, wherein the rendering component **602** detects the reference surface in the 3D space captured in the camera feed. At operation **901**, the event detection component **610** detects a triggering event. The triggering event may, for example, be based on geolocation data from a location component of the computing device. As an example, the detecting of the triggering event may include detecting when the computing device is within a predefined distance of the geographic location. As an additional example, the triggering event may be based on temporal factors and thus, the detecting the triggering event may include detecting a particular date or time. As yet another example, the triggering event may be based on weather conditions, and thus, the detecting of the triggering event may include detecting certain weather condition (e.g., snow, rain, wind, etc.).

[0102] The operation **905** may be performed prior to operation **906**, where the rendering component **602** applies a visual effect to the image mask. At operation **905**, the rendering component **602** selects the visual effect to be subsequently applied to the image mask. In some embodiments, the rendering component **602** selects the visual effect based on previous user input that specified a particular visual effect.

[0103] In some embodiments, the triggering event detected at operation **901** is associated with a particular visual effect, and thus, the rendering component **602** may select the graphical event based on the triggering event. For example, a particular visual effect may be associated with a particular geographic location, and when the event detection component **610** detects the computing device being within the predefined distance of the geographic location, the rendering component **602** selects the visual effect associated with the geographic location.

[0104] As shown in FIG. **11**, the method **900** may, in some embodiments, also include operations **1102**, **1104**, and **1106**. One or more of the operations **1102**, **1104**, and **1106** may be performed as part of (e.g., a precursor task, a subroutine, or a portion) operation **904** where the rendering component **602** applies the image mask to the reference surface. The operations **1102**, **1104**, and **1106** are described below in reference to a single image from the camera feed, though it shall be appreciated that the operations **1102**, **1104**, and **1106** may be repeated for each image of the camera feed to achieve the dynamic application of the image mask to the reference surface as depicted in the camera feed as a whole.

[0105] At operation **1102**, the rendering component **602** determines the boundaries of the reference surface within the 3D space depicted in an image of the camera feed. In determining the boundaries of the reference surface, the

rendering component **602** may, for example, employ one of many known edge detection techniques. Further details regarding the determining of the boundaries of the reference surface, according to some embodiments, are discussed below in reference to FIGS. **12** and **13**.

[0106] At operation **1104**, the rendering component **602** classifies pixels in the image that are inside the boundaries of the references surface according to a first class. At operation **1106**, the rendering component **602** classifies pixels in the image that are outside the boundaries of the references surface according to a second class. The pixels classified according to the first class form the image mask for the reference surface.

[0107] It shall be appreciated that although FIG. **11** illustrates the operation of determining the boundaries of the reference surface (operation **1102**) as being separate and distinct from operations **1104** and **1106**, the determination of the boundaries of the reference surface may, in some embodiments, be combined and performed as part of the operations **1104** and **1106**.

[0108] As shown in FIG. **12**, the method **900** may, in some embodiments, further include operations **1202**, **1204**, **1206**, **1208**, **1210**, and **1212**. The operations **1202**, **1204**, **1206**, **1208**, **1210**, and **1212** may, in some embodiments, be performed as part of (e.g., a precursor task, a subroutine, or a portion) operation **1004** where the rendering component **602** applies the image mask to reference surface. In some embodiments, the operations **1202**, **1204**, **1206**, **1208**, **1210**, and **1212** may be performed specifically as part of (e.g., a precursor task, a subroutine, or a portion) operation **1202** where the rendering component **602** determines the boundaries of the reference surface.

[0109] At operation **1202**, the rendering component **602** obtains a set of points from the multiple images of the camera feed. The obtaining of the set of points may including sampling a set of pixels randomly selected from the camera feed.

[0110] At operation **1204**, the rendering component **602** associates a first subset of the set of points with a first plane in the 3D space depicted in the image. At operations **1206**-**1208**, the rendering component **602** respectively associates a second-Nth subset of the set of points with a second-Nth plane in 3D space depicted in the image. The rendering component **602** may associate each subset of points with a corresponding plane in the 3D space based on a determined geometry of the 3D space, sensor data from one or more sensors of the computing device (e.g., a gyroscope and accelerometer), location data from one or more location components (e.g., compass and global positioning system (GPS)), or various combinations of both. For example, the associating of the set of points with a corresponding plane may include determining a location and orientation of the camera with respect to the plane based in part on sensor data (e.g., gyroscope and accelerometer) and mapping pixels locations in the images to locations in the 3D space based in part on location data.

[0111] At operation **1210**, the rendering component **602** determines the first plane corresponds to the detected reference surface. At operation **1212**, the rendering component **602** identifies the boundaries of the first plane and in doing so, the rendering component **602** determines the boundaries of the reference surface.

[0112] As shown in FIG. **13**, the method **1000** may, in some embodiments, further include operations **1302**, **1304**, and **1306**. The operations **1302**, **1304**, and **1306** may, in some embodiments, be performed as part of (e.g., a precursor task, a subroutine, or a portion) operation **1004** where the rendering component **602** applies the image mask to reference surface. In some embodiments, the operations **1302**, **1304**, and **1306** may be performed specifically as part of (e.g., a precursor task, a subroutine, or a portion) operation **1202** where the rendering component **602** determines the boundaries of the reference surface. The operations **1302**, **1304**, and **1306** are described below in reference to a single image from the camera feed, though it shall be appreciated that the operations **1302**, **1304**, and **1306** may be repeated for each image of the camera feed to achieve the dynamic application of the image mask to the reference surface as depicted in the camera feed as a whole.

[0113] At operation **1302**, the rendering component **602** identifies regions of similar color in an image of the camera feed. The identifying of the regions of similar color may include associating groupings of pixels in the image based on pixel color values. More specifically, the rendering component **602** may associate groupings of pixels based on pixels within the groupings having pixel color values that do not exceed a threshold standard deviation.

[0114] At operation **1304**, the rendering component **602** determines which region of similar color corresponds to the detected reference surface. At operation **1306**, the rendering component **602** identifies the boundaries of the region of similar color that corresponds to the reference surface thereby determining the boundaries of the reference surface.

[0115] FIGS. **14**A and **14**B are interface diagrams illustrating aspects of interface provided by the virtual rendering system **210**, according to example embodiments. With reference to FIG. **14**A, a camera feed **1400** is shown. The camera feed **1400** may be displayed on a display of a computing device (e.g., client device **102**) and although FIG. **14**A only illustrates a single image, the camera feed **1400** may comprise a sequence of images (e.g., a video) produced by a camera of the computing device. Although FIG. **14**A illustrates the camera feed **1400** being displayed in isolation, the camera feed **1400** may be presented within or as part of a user interface element that is displayed among other user interface elements that facilitate functionality that allows a user to interact with the camera feed **1400** in various ways.

[0116] A 3D real-world environment is depicted within the camera feed **1400**. In particular, the 3D real-world environment depicted within the camera feed includes a sidewalk surface **1402**. The virtual rendering system **210** may analyze the camera feed **1400**, utilizing other device inputs such as a gyroscope, accelerometer, and compass to identify the sidewalk surface **1402**, and dynamically apply an image mask to the sidewalk surface **1402**, in accordance with any one of the methodologies described above. The virtual rendering system **210** may modify the sidewalk surface **1402** as presented in the camera feed **1400** by applying a visual effect to the image mask corresponding to the sidewalk surface **1402**. The applying of the visual effect to the image mask causes a modified surface to be rendered in a presentation of the camera feed **1400**.

[0117] For example, FIG. **14**B illustrates a modified surface **1404** presented within the camera feed **1400**. The modified surface **1404** corresponds to the sidewalk surface **1402** with an applied visual effect. In particular, the modified

surface **1402** includes an animation that makes the sidewalk surface **1402** appear in the camera feed **1400** to be flowing water rather than a sidewalk.

[0118] FIG. **15** is a block diagram illustrating an example software architecture **1506**, which may be used in conjunction with various hardware architectures herein described. FIG. **15** is a non-limiting example of a software architecture and it will be appreciated that many other architectures may be implemented to facilitate the functionality described herein. The software architecture **1506** may execute on hardware such as machine **1600** of FIG. **16** that includes, among other things, processors **1604**, memory **1614**, and input/output (I/O) components **1618**. A representative hardware layer **1552** is illustrated and can represent, for example, the machine **1600** of FIG. **16**. The representative hardware layer **1552** includes a processing unit **1554** having associated executable instructions **1504**. Executable instructions **1504** represent the executable instructions of the software architecture **1506**, including implementation of the methods, components and so forth described herein. The hardware layer **1552** also includes memory and/or storage modules memory/storage **1556**, which also have executable instructions **1504**. The hardware layer **1552** may also comprise other hardware **1558**.

[0119] In the example architecture of FIG. **15**, the software architecture **1506** may be conceptualized as a stack of layers where each layer provides particular functionality. For example, the software architecture **1506** may include layers such as an operating system **1502**, libraries **1520**, applications **1516**, frameworks/middleware **1518**, and a presentation layer **1514**. Operationally, the applications **1516** and/or other components within the layers may invoke API calls **1508** through the software stack and receive a response **1512** as in response to the API calls **1508**. The layers illustrated are representative in nature and not all software architectures have all layers. For example, some mobile or special purpose operating systems may not provide a frameworks/middleware **1518**, while others may provide such a layer. Other software architectures may include additional or different layers.

[0120] The operating system **1502** may manage hardware resources and provide common services. The operating system **1502** may include, for example, a kernel **1522**, services **1524**, and drivers **1526**. The kernel **1522** may act as an abstraction layer between the hardware and the other software layers. For example, the kernel **1522** may be responsible for memory management, processor management (e.g., scheduling), component management, networking, security settings, and so on. The services **1524** may provide other common services for the other software layers. The drivers **1526** are responsible for controlling or interfacing with the underlying hardware. For instance, the drivers **1526** include display drivers, camera drivers, Bluetooth® drivers, flash memory drivers, serial communication drivers (e.g., Universal Serial Bus (USB) drivers), Wi-Fi® drivers, audio drivers, power management drivers, and so forth depending on the hardware configuration.

[0121] The libraries **1520** provide a common infrastructure that is used by the applications **1516** and/or other components and/or layers. The libraries **1520** provide functionality that allows other software components to perform tasks in an easier fashion than to interface directly with the underlying operating system **1502** functionality (e.g., kernel **1522**, services **1524** and/or drivers **1526**). The libraries **1520** may include system libraries **1544** (e.g., C standard library) that may provide functions such as memory allocation functions, string manipulation functions, mathematical functions, and the like. In addition, the libraries **1520** may include API libraries **1546** such as media libraries (e.g., libraries to support presentation and manipulation of various media format such as MPREG4, H.264, MP3, AAC, AMR, JPG, PNG), graphics libraries (e.g., an OpenGL framework that may be used to render two-dimensional and 3D in a graphic content on a display), database libraries (e.g., SQLite that may provide various relational database functions), web libraries (e.g., WebKit that may provide web browsing functionality), and the like. The libraries **1520** may also include a wide variety of other libraries **1548** to provide many other APIs to the applications **1516** and other software components/modules.

[0122] The frameworks/middleware **1518** (also sometimes referred to as middleware) provide a higher-level common infrastructure that may be used by the applications **1516** and/or other software components/modules. For example, the frameworks/middleware **1518** may provide various graphic user interface (GUI) functions, high-level resource management, high-level location services, and so forth. The frameworks/middleware **1518** may provide a broad spectrum of other APIs that may be utilized by the applications **1516** and/or other software components/modules, some of which may be specific to a particular operating system **1502** or platform.

[0123] The applications **1516** include built-in applications **1538** and/or third-party applications **1540**. Examples of representative built-in applications **1538** may include, but are not limited to, a contacts application, a browser application, a book reader application, a location application, a media application, a messaging application, and/or a game application. Third-party applications **1540** may include an application developed using the ANDROID™ or IOS™ software development kit (SDK) by an entity other than the vendor of the particular platform, and may be mobile software running on a mobile operating system such as IOS™, ANDROID™, WINDOWS® Phone, or other mobile operating systems. The third-party applications **1540** may invoke the API calls **1508** provided by the mobile operating system (such as operating system **1502**) to facilitate functionality described herein.

[0124] The applications **1516** may use built in operating system functions (e.g., kernel **1522**, services **1524**, and/or drivers **1526**), libraries **1520**, and frameworks/middleware **1518** to create user interfaces to interact with users of the system. Alternatively, or additionally, in some systems interactions with a user may occur through a presentation layer, such as presentation layer **1514**. In these systems, the application/component "logic" can be separated from the aspects of the application/component that interact with a user.

[0125] FIG. **16** is a block diagram illustrating components of a machine **1600**, according to some example embodiments, able to read instructions from a machine-readable medium (e.g., a machine-readable storage medium) and perform any one or more of the methodologies discussed herein. Specifically, FIG. **16** shows a diagrammatic representation of the machine **1600** in the example form of a computer system, within which instructions **1610** (e.g., software, a program, an application, an applet, an app, or other executable code) for causing the machine **1600** to

perform any one or more of the methodologies discussed herein may be executed. As such, the instructions **1610** may be used to implement modules or components described herein. The instructions **1610** transform the general, non-programmed machine **1600** into a particular machine **1600** programmed to carry out the described and illustrated functions in the manner described. In alternative embodiments, the machine **1600** operates as a standalone device or may be coupled (e.g., networked) to other machines. In a networked deployment, the machine **1600** may operate in the capacity of a server machine or a client machine in a server-client network environment, or as a peer machine in a peer-to-peer (or distributed) network environment. The machine **1600** may comprise, but not be limited to, a server computer, a client computer, a personal computer (PC), a tablet computer, a laptop computer, a netbook, a set-top box (STB), a personal digital assistant (PDA), an entertainment media system, a cellular telephone, a smart phone, a mobile device, a wearable device (e.g., a smart watch), a smart home device (e.g., a smart appliance), other smart devices, a web appliance, a network router, a network switch, a network bridge, or any machine capable of executing the instructions **1610**, sequentially or otherwise, that specify actions to be taken by machine **1600**. Further, while only a single machine **1600** is illustrated, the term "machine" shall also be taken to include a collection of machines that individually or jointly execute the instructions **1610** to perform any one or more of the methodologies discussed herein.

[0126] The machine **1600** may include processors **1604**, memory memory/storage **1606**, and I/O components **1618**, which may be configured to communicate with each other such as via a bus **1602**. In an example embodiment, the processors **1604** (e.g., a central processing unit (CPU), a reduced instruction set computing (RISC) processor, a complex instruction set computing (CISC) processor, a graphics processing unit (GPU), a digital signal processor (DSP), an application-specific integrated circuit (ASIC), a radio-frequency integrated circuit (RFIC), another processor, or any suitable combination thereof) may include, for example, a processor **1608** and a processor **1612** that may execute the instructions **1610**. The term "processor" is intended to include multi-core processors **1604** that may comprise two or more independent processors (sometimes referred to as "cores") that may execute instructions contemporaneously. Although FIG. **16** shows multiple processors, the machine **1600** may include a single processor with a single core, a single processor with multiple cores (e.g., a multi-core processor), multiple processors with a single core, multiple processors with multiple cores, or any combination thereof.

[0127] The memory/storage **1606** may include a memory **1614**, such as a main memory, or other memory storage, and a storage unit **1616**, both accessible to the processors **1604** such as via the bus **1602**. The storage unit **1616** and memory **1614** store the instructions **1610** embodying any one or more of the methodologies or functions described herein. The instructions **1610** may also reside, completely or partially, within the memory **1614**, within the storage unit **1616**, within at least one of the processors **1604** (e.g., within the processor's cache memory), or any suitable combination thereof, during execution thereof by the machine **1600**. Accordingly, the memory **1614**, the storage unit **1616**, and the memory of processors **1604** are examples of machine-readable media.

[0128] The I/O components **1618** may include a wide variety of components to receive input, provide output, produce output, transmit information, exchange information, capture measurements, and so on. The specific I/O components **1618** that are included in a particular machine **1600** will depend on the type of machine. For example, portable machines such as mobile phones will likely include a touch input device or other such input mechanisms, while a headless server machine will likely not include such a touch input device. It will be appreciated that the I/O components **1618** may include many other components that are not shown in FIG. **16**. The I/O components **1618** are grouped according to functionality merely for simplifying the following discussion and the grouping is in no way limiting. In various example embodiments, the I/O components **1618** may include output components **1626** and input components **1628**. The output components **1626** may include visual components (e.g., a display such as a plasma display panel (PDP), a light emitting diode (LED) display, a liquid crystal display (LCD), a projector, or a cathode ray tube (CRT)), acoustic components (e.g., speakers), haptic components (e.g., a vibratory motor, resistance mechanisms), other signal generators, and so forth. The input components **1628** may include alphanumeric input components (e.g., a keyboard, a touch screen configured to receive alphanumeric input, a photo-optical keyboard, or other alphanumeric input components), point based input components (e.g., a mouse, a touchpad, a trackball, a joystick, a motion sensor, or other pointing instrument), tactile input components (e.g., a physical button, a touch screen that provides location and/or force of touches or touch gestures, or other tactile input components), audio input components (e.g., a microphone), and the like.

[0129] In further example embodiments, the I/O components **1618** may include biometric components **1630**, motion components **1634**, environmental components **1636**, or position components **1638** among a wide array of other components. For example, the biometric components **1630** may include components to detect expressions (e.g., hand expressions, facial expressions, vocal expressions, body gestures, or eye tracking), measure biosignals (e.g., blood pressure, heart rate, body temperature, perspiration, or brain waves), identify a person (e.g., voice identification, retinal identification, facial identification, fingerprint identification, or electroencephalogram based identification), and the like. The motion components **1634** may include acceleration sensor components (e.g., accelerometer), gravitation sensor components, rotation sensor components (e.g., gyroscope), and so forth. The environment components **1636** may include, for example, illumination sensor components (e.g., photometer), temperature sensor components (e.g., one or more thermometer that detect ambient temperature), humidity sensor components, pressure sensor components (e.g., barometer), acoustic sensor components (e.g., one or more microphones that detect background noise), proximity sensor components (e.g., infrared sensors that detect nearby objects), gas sensors (e.g., gas detection sensors to detection concentrations of hazardous gases for safety or to measure pollutants in the atmosphere), or other components that may provide indications, measurements, or signals corresponding to a surrounding physical environment. The position components **1638** may include location sensor components (e.g., a GPS receiver component), altitude sensor components (e.g., altimeters or barometers that detect air pressure from

which altitude may be derived), orientation sensor components (e.g., magnetometers), and the like.

[0130] Communication may be implemented using a wide variety of technologies. The I/O components **1618** may include communication components **1640** operable to couple the machine **1600** to a network **1632** or devices **1620** via coupling **1624** and coupling **1622**, respectively. For example, the communication components **1640** may include a network interface component or other suitable device to interface with the network **1632**. In further examples, communication components **1640** may include wired communication components, wireless communication components, cellular communication components, Near Field Communication (NFC) components, Bluetooth® components (e.g., Bluetooth® Low Energy), Wi-Fi® components, and other communication components to provide communication via other modalities. The devices **1620** may be another machine or any of a wide variety of peripheral devices (e.g., a peripheral device coupled via a USB).

[0131] Moreover, the communication components **1640** may detect identifiers or include components operable to detect identifiers. For example, the communication components **1640** may include Radio Frequency Identification (RFID) tag reader components, NFC smart tag detection components, optical reader components (e.g., an optical sensor to detect one-dimensional bar codes such as Universal Product Code (UPC) bar code, multi-dimensional bar codes such as Quick Response (QR) code, Aztec code, Data Matrix, Dataglyph, MaxiCode, PDF417, Ultra Code, UCC RSS-2D bar code, and other optical codes), or acoustic detection components (e.g., microphones to identify tagged audio signals). In addition, a variety of information may be derived via the communication components **1640**, such as, location via Internet Protocol (IP) geo-location, location via Wi-Fi® signal triangulation, location via detecting a NFC beacon signal that may indicate a particular location, and so forth.

GLOSSARY

[0132] "CARRIER SIGNAL" in this context refers to any intangible medium that is capable of storing, encoding, or carrying instructions for execution by the machine, and includes digital or analog communications signals or other intangible medium to facilitate communication of such instructions. Instructions may be transmitted or received over the network using a transmission medium via a network interface device and using any one of a number of well-known transfer protocols.

[0133] "CLIENT DEVICE" in this context refers to any machine that interfaces to a communications network to obtain resources from one or more server systems or other client devices. A client device may be, but is not limited to, a mobile phone, desktop computer, laptop, PDAs, smart phones, tablets, ultra books, netbooks, laptops, multi-processor systems, microprocessor-based or programmable consumer electronics, game consoles, set-top boxes, or any other communication device that a user may use to access a network.

[0134] "COMMUNICATIONS NETWORK" in this context refers to one or more portions of a network that may be an ad hoc network, an intranet, an extranet, a virtual private network (VPN), a local area network (LAN), a wireless LAN (WLAN), a wide area network (WAN), a wireless WAN (WWAN), a metropolitan area network (MAN), the Internet, a portion of the Internet, a portion of the Public Switched Telephone Network (PSTN), a plain old telephone service (POTS) network, a cellular telephone network, a wireless network, a Wi-Fi® network, another type of network, or a combination of two or more such networks. For example, a network or a portion of a network may include a wireless or cellular network and the coupling may be a Code Division Multiple Access (CDMA) connection, a Global System for Mobile communications (GSM) connection, or other type of cellular or wireless coupling. In this example, the coupling may implement any of a variety of types of data transfer technology, such as Single Carrier Radio Transmission Technology (1xRTT), Evolution-Data Optimized (EVDO) technology, General Packet Radio Service (GPRS) technology, Enhanced Data rates for GSM Evolution (EDGE) technology, third Generation Partnership Project (3GPP) including 3G, fourth generation wireless (4G) networks, Universal Mobile Telecommunications System (UMTS), High Speed Packet Access (HSPA), Worldwide Interoperability for Microwave Access (WiMAX), Long Term Evolution (LTE) standard, others defined by various standard setting organizations, other long range protocols, or other data transfer technology.

[0135] "EMPHEMERAL MESSAGE" in this context refers to a message that is accessible for a time-limited duration. An ephemeral message may be a text, an image, a video and the like. The access time for the ephemeral message may be set by the message sender. Alternatively, the access time may be a default setting or a setting specified by the recipient. Regardless of the setting technique, the message is transitory.

[0136] "MACHINE-READABLE MEDIUM" in this context refers to a component, device or other tangible media able to store instructions and data temporarily or permanently and may include, but is not be limited to, random-access memory (RAM), read-only memory (ROM), buffer memory, flash memory, optical media, magnetic media, cache memory, other types of storage (e.g., Erasable Programmable Read-Only Memory (EEPROM)) and/or any suitable combination thereof. The term "machine-readable medium" should be taken to include a single medium or multiple media (e.g., a centralized or distributed database, or associated caches and servers) able to store instructions. The term "machine-readable medium" shall also be taken to include any medium, or combination of multiple media, that is capable of storing instructions (e.g., code) for execution by a machine, such that the instructions, when executed by one or more processors of the machine, cause the machine to perform any one or more of the methodologies described herein. Accordingly, a "machine-readable medium" refers to a single storage apparatus or device, as well as "cloud-based" storage systems or storage networks that include multiple storage apparatus or devices. The term "machine-readable medium" excludes signals per se.

[0137] "COMPONENT" in this context refers to a device, physical entity, or logic having boundaries defined by function or subroutine calls, branch points, APIs, or other technologies that provide for the partitioning or modularization of particular processing or control functions. Components may be combined via their interfaces with other components to carry out a machine process. A component may be a packaged functional hardware unit designed for use with other components and a part of a program that usually performs a particular function of related functions. Compo-

nents may constitute either software components (e.g., code embodied on a machine-readable medium) or hardware components. A "hardware component" is a tangible unit capable of performing certain operations and may be configured or arranged in a certain physical manner. In various example embodiments, one or more computer systems (e.g., a standalone computer system, a client computer system, or a server computer system) or one or more hardware components of a computer system (e.g., a processor or a group of processors) may be configured by software (e.g., an application or application portion) as a hardware component that operates to perform certain operations as described herein. A hardware component may also be implemented mechanically, electronically, or any suitable combination thereof. For example, a hardware component may include dedicated circuitry or logic that is permanently configured to perform certain operations. A hardware component may be a special-purpose processor, such as a Field-Programmable Gate Array (FPGA) or an ASIC. A hardware component may also include programmable logic or circuitry that is temporarily configured by software to perform certain operations. For example, a hardware component may include software executed by a general-purpose processor or other programmable processor. Once configured by such software, hardware components become specific machines (or specific components of a machine) uniquely tailored to perform the configured functions and are no longer general-purpose processors. It will be appreciated that the decision to implement a hardware component mechanically, in dedicated and permanently configured circuitry, or in temporarily configured circuitry (e.g., configured by software) may be driven by cost and time considerations. Accordingly, the phrase "hardware component" (or "hardware-implemented component") should be understood to encompass a tangible entity, be that an entity that is physically constructed, permanently configured (e.g., hardwired), or temporarily configured (e.g., programmed) to operate in a certain manner or to perform certain operations described herein. Considering embodiments in which hardware components are temporarily configured (e.g., programmed), each of the hardware components need not be configured or instantiated at any one instance in time. For example, where a hardware component comprises a general-purpose processor configured by software to become a special-purpose processor, the general-purpose processor may be configured as respectively different special-purpose processors (e.g., comprising different hardware components) at different times. Software accordingly configures a particular processor or processors, for example, to constitute a particular hardware component at one instance of time and to constitute a different hardware component at a different instance of time. Hardware components can provide information to, and receive information from, other hardware components. Accordingly, the described hardware components may be regarded as being communicatively coupled. Where multiple hardware components exist contemporaneously, communications may be achieved through signal transmission (e.g., over appropriate circuits and buses) between or among two or more of the hardware components. In embodiments in which multiple hardware components are configured or instantiated at different times, communications between such hardware components may be achieved, for example, through the storage and retrieval of information in memory structures to which the multiple hardware components have access. For example, one hardware component may perform an operation and store the output of that operation in a memory device to which it is communicatively coupled. A further hardware component may then, at a later time, access the memory device to retrieve and process the stored output. Hardware components may also initiate communications with input or output devices, and can operate on a resource (e.g., a collection of information). The various operations of example methods described herein may be performed, at least partially, by one or more processors that are temporarily configured (e.g., by software) or permanently configured to perform the relevant operations. Whether temporarily or permanently configured, such processors may constitute processor-implemented components that operate to perform one or more operations or functions described herein. As used herein, "processor-implemented component" refers to a hardware component implemented using one or more processors. Similarly, the methods described herein may be at least partially processor-implemented, with a particular processor or processors being an example of hardware. For example, at least some of the operations of a method may be performed by one or more processors or processor-implemented components. Moreover, the one or more processors may also operate to support performance of the relevant operations in a "cloud computing" environment or as a "software as a service" (SaaS). For example, at least some of the operations may be performed by a group of computers (as examples of machines including processors), with these operations being accessible via a network (e.g., the Internet) and via one or more appropriate interfaces (e.g., an API). The performance of certain of the operations may be distributed among the processors, not only residing within a single machine, but deployed across a number of machines. In some example embodiments, the processors or processor-implemented components may be located in a single geographic location (e.g., within a home environment, an office environment, or a server farm). In other example embodiments, the processors or processor-implemented components may be distributed across a number of geographic locations.

[0138] "PROCESSOR" in this context refers to any circuit or virtual circuit (a physical circuit emulated by logic executing on an actual processor) that manipulates data values according to control signals (e.g., "commands", "op codes", "machine code", etc.) and which produces corresponding output signals that are applied to operate a machine. A processor may, for example, be a CPU, a RISC processor, a CISC processor, a GPU, a DSP, an ASIC, a RFIC), or any combination thereof. A processor may further be a multi-core processor having two or more independent processors (sometimes referred to as "cores") that may execute instructions contemporaneously.

[0139] "TIMESTAMP" in this context refers to a sequence of characters or encoded information identifying when a certain event occurred, for example giving date and time of day, sometimes accurate to a small fraction of a second.

What is claimed is:

1. A system comprising:

one or more processors; and

memory storing instructions that, when executed by the one or more processors, cause the system to perform operations comprising:

tracking, via a first tracking system, a location for a visual effect applied to a real-world surface using first tracking indicia generated by the first tracking system;

storing the first tracking indicia in a tracking matrix associated with the visual effect; and

in response to detecting an interruption of the first tracking indicia, switching from tracking the location for the visual effect via the first tracking system to tracking the location for the visual effect via a second tracking system using second tracking indicia, the switching based on the tracking matrix.

2. The system of claim **1**, the operations further comprising:

detecting the first tracking indicia is available after the interruption of the first tracking indicia;

storing a difference between a first value for the first tracking indicia and a second value for the first tracking indicia in the tracking matrix, the first value associated with when the first tracking indicia is available, the second value associated with the interruption of the first tracking indicia; and

adjusting the location for the visual effect applied to the real-world surface based on the difference.

3. The system of claim **1**, wherein switching from tracking the location for the visual effect via the first tracking system to tracking the location for the visual effect via the second tracking system comprises:

replacing a first orientation for the visual effect based on the first tracking indicia in the tracking matrix with a second orientation for the visual effect based on the second tracking indicia.

4. The system of claim **1**, wherein switching from tracking the location for the visual effect via the first tracking system to tracking the location for the visual effect via the second tracking system comprises:

extracting, from the tracking matrix, translation indicia from the first tracking indicia generated by the first tracking system prior to the interruption; and

tracking the location for the visual effect based on the translation indicia and rotation indicia from the second tracking indicia.

5. The system of claim **1**, wherein the first tracking indicia is based on a visual feature of an environment tracked by the first tracking system, the interruption of the first tracking indicia comprising an obstruction of the visual feature.

6. The system of claim **1**, wherein the second tracking indicia comprises at least one of roll, pitch, or yaw.

7. The system of claim **1**, wherein the first tracking system provides tracking in six degrees of freedom and the second tracking system provides tracking in three degrees of freedom.

8. The system of claim **1**, wherein the tracking matrix comprises translation indicia and rotation indicia associated with the visual effect.

9. A method comprising:

tracking, via a first tracking system, a location for a visual effect applied to a real-world surface using first tracking indicia generated by the first tracking system;

storing the first tracking indicia in a tracking matrix associated with the visual effect; and

in response to detecting an interruption of the first tracking indicia, switching from tracking the location for the visual effect via the first tracking system to tracking the location for the visual effect via a second tracking system using second tracking indicia, the switching based on the tracking matrix.

10. The method of claim **9**, further comprising:

detecting the first tracking indicia is available after the interruption of the first tracking indicia;

storing a difference between a first value for the first tracking indicia and a second value for the first tracking indicia in the tracking matrix, the first value associated with when the first tracking indicia is available, the second value associated with the interruption of the first tracking indicia; and

adjusting the location for the visual effect applied to the real-world surface based on the difference.

11. The method of claim **9**, wherein switching from tracking the location for the visual effect via the first tracking system to tracking the location for the visual effect via the second tracking system comprises:

replacing a first orientation for the visual effect based on the first tracking indicia in the tracking matrix with a second orientation for the visual effect based on the second tracking indicia.

12. The method of claim **9**, wherein switching from tracking the location for the visual effect via the first tracking system to tracking the location for the visual effect via the second tracking system comprises:

extracting, from the tracking matrix, translation indicia from the first tracking indicia generated by the first tracking system prior to the interruption; and

tracking the location for the visual effect based on the translation indicia and rotation indicia from the second tracking indicia.

13. The method of claim **9**, wherein the first tracking indicia is based on a visual feature of an environment tracked by the first tracking system, the interruption of the first tracking indicia comprising an obstruction of the visual feature.

14. The method of claim **9**, wherein the second tracking indicia comprises at least one of roll, pitch, or yaw.

15. The method of claim **9**, wherein the first tracking system provides tracking in six degrees of freedom and the second tracking system provides tracking in three degrees of freedom.

16. The method of claim **9**, wherein the tracking matrix comprises translation indicia and rotation indicia associated with the visual effect.

17. A non-transitory machine-readable medium storing instructions which, when executed by a machine, cause the machine to perform operations comprising:

tracking, via a first tracking system, a location for a visual effect applied to a real-world surface using first tracking indicia generated by the first tracking system;

storing the first tracking indicia in a tracking matrix associated with the visual effect; and

in response to detecting an interruption of the first tracking indicia, switching from tracking the location for the visual effect via the first tracking system to tracking the location for the visual effect via a second tracking system using second tracking indicia, the switching based on the tracking matrix.

18. The non-transitory machine-readable medium of claim **17**, the operations further comprising:

detecting the first tracking indicia is available after the interruption of the first tracking indicia;

storing a difference between a first value for the first tracking indicia and a second value for the first tracking indicia in the tracking matrix, the first value associated with when the first tracking indicia is available, the second value associated with the interruption of the first tracking indicia; and

adjusting the location for the visual effect applied to the real-world surface based on the difference.

19. The non-transitory machine-readable medium of claim **17**, wherein switching from tracking the location for the visual effect via the first tracking system to tracking the location for the visual effect via the second tracking system comprises:

replacing a first orientation for the visual effect based on the first tracking indicia in the tracking matrix with a second orientation for the visual effect based on the second tracking indicia.

20. The non-transitory machine-readable medium of claim **17**, wherein switching from tracking the location for the visual effect via the first tracking system to tracking the location for the visual effect via the second tracking system comprises:

extracting, from the tracking matrix, translation indicia from the first tracking indicia generated by the first tracking system prior to the interruption; and

tracking the location for the visual effect based on the translation indicia and rotation indicia from the second tracking indicia.

* * * * *